THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Jiacheng Li, Ph.D.

B-Trees

CS 201: Data Structures and Algorithms

Jiacheng Li, Ph.D.

Desktop Configuration

Component	Specification
CPU	Intel Core i9-14900K (24 cores, 32 threads)
Motherboard	ASUS ROG Maximus Z790 Hero
RAM	64GB DDR5 6000MHz (Corsair Dominator Platinum / G.Skill Trident Z5)
GPU	NVIDIA RTX 4090 24GB (AI/ML, CUDA support)
Storage 1	2TB NVMe SSD (Samsung 990 Pro) - OS & software
Storage 2	10TB HDD (Seagate Exos X18) - Large data storage
Power Supply	1200W Platinum (Corsair AX1200i)
Cooling	Custom Liquid Cooling or AIO Cooler (Corsair iCUE H150i Elite)
Case	Lian Li O11 Dynamic XL
Operating System	Windows 11 Pro + Ubuntu (Dual Boot)

- Main memory: RAM.
- It's used for temporary data storage.

Desktop Configuration

Component	Specification
CPU	Intel Core i9-14900K (24 cores, 32 threads)
Motherboard	ASUS ROG Maximus Z790 Hero
RAM	64GB DDR5 6000MHz (Corsair Dominator Platinum / G.Skill Trident Z5)
GPU	NVIDIA RTX 4090 24GB (AI/ML, CUDA support)
Storage 1	2TB NVMe SSD (Samsung 990 Pro) - OS & software
Storage 2	10TB HDD (Seagate Exos X18) - Large data storage
Power Supply	1200W Platinum (Corsair AX1200i)
Cooling	Custom Liquid Cooling or AIO Cooler (Corsair iCUE H150i Elite)
Case	Lian Li O11 Dynamic XL
Operating System	Windows 11 Pro + Ubuntu (Dual Boot)

- Disk Storage: Data is permanently stored.
  - Hard Disk Drive (HDD)
  - Solid-State Drive (SSD)

Desktop Configuration

Component	Specification
CPU	Intel Core i9-14900K (24 cores, 32 threads)
Motherboard	ASUS ROG Maximus Z790 Hero
RAM	64GB DDR5 6000MHz (Corsair Dominator Platinum / G.Skill Trident Z5)
GPU	NVIDIA RTX 4090 24GB (AI/ML, CUDA support)
Storage 1	2TB NVMe SSD (Samsung 990 Pro) - OS & software
Storage 2	10TB HDD (Seagate Exos X18) - Large data storage
Power Supply	1200W Platinum (Corsair AX1200i)
Cooling	Custom Liquid Cooling or AIO Cooler (Corsair iCUE H150i Elite)
Case	Lian Li O11 Dynamic XL
Operating System	Windows 11 Pro + Ubuntu (Dual Boot)

- When you open a program or access a file, the operating system (OS) loads the necessary data and instructions from the disk storage into RAM.

Desktop Configuration

Component	Specification
CPU	Intel Core i9-14900K (24 cores, 32 threads)
Motherboard	ASUS ROG Maximus Z790 Hero
RAM	64GB DDR5 6000MHz (Corsair Dominator Platinum / G.Skill Trident Z5)
GPU	NVIDIA RTX 4090 24GB (AI/ML, CUDA support)
Storage 1	2TB NVMe SSD (Samsung 990 Pro) - OS & software
Storage 2	10TB HDD (Seagate Exos X18) - Large data storage
Power Supply	1200W Platinum (Corsair AX1200i)
Cooling	Custom Liquid Cooling or AIO Cooler (Corsair iCUE H150i Elite)
Case	Lian Li O11 Dynamic XL
Operating System	Windows 11 Pro + Ubuntu (Dual Boot)

- Once the data and instructions are in RAM, the CPU can access them quickly and efficiently to perform the necessary operations.
- The CPU cannot work directly with data on disk; it can only work with data in RAM.

Example - Opening a Large Image File

1. You click on a **2GB image file** stored on your **HDD/SSD**.
2. The OS checks if the image is already in **RAM (cache)**.
3. If not, the OS sends a **DISK-READ request** to the HDD/SSD.
4. The disk finds the file and loads **a large portion of the image** into RAM.
5. Your image viewer software processes the image **from RAM** (not directly from disk).
6. If you edit and save the file, the **updated data is written back to disk (DISK-WRITE)**.

Desktop Configuration

Component	Specification
CPU	Intel Core i9-14900K (24 cores, 32 threads)
Motherboard	ASUS ROG Maximus Z790 Hero
RAM	64GB DDR5 6000MHz (Corsair Dominator Platinum / G.Skill Trident Z5)
GPU	NVIDIA RTX 4090 24GB (AI/ML, CUDA support)
Storage 1	2TB NVMe SSD (Samsung 990 Pro) - OS & software
Storage 2	10TB HDD (Seagate Exos X18) - Large data storage
Power Supply	1200W Platinum (Corsair AX1200i)
Cooling	Custom Liquid Cooling or AIO Cooler (Corsair iCUE H150i Elite)
Case	Lian Li O11 Dynamic XL
Operating System	Windows 11 Pro + Ubuntu (Dual Boot)

Indexing Structures for Fast Lookups

- An indexing structure is a specialized data structure designed to speed up the retrieval of data from a larger dataset. In other words, it acts like a roadmap or table of contents, helping systems locate and access information quickly without scanning every item.
- The most common indexing structures used in disk storage are:
  - B-Trees and B+ Trees.
  - Hash Indexing.

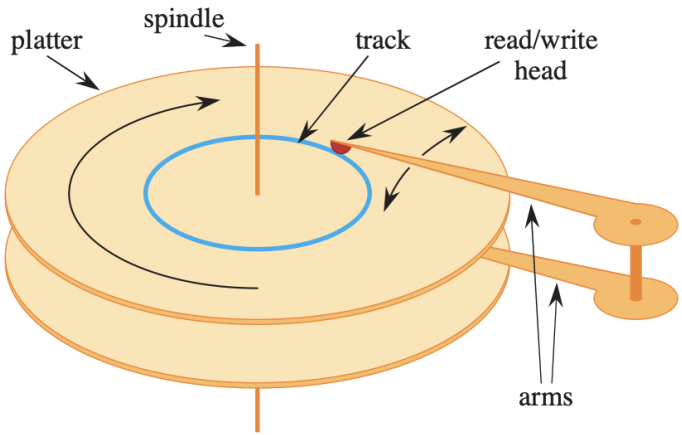


Figure: A typical magnetic disk drive.

B-Trees

- High Branching Factor Keeps Tree Shallow
  - Branching Factor: The number of children per node.
  - B-trees have a high branching factor (e.g., 50–2000 children per node), keeping the tree shallow.
  - Shallow trees mean fewer disk reads.

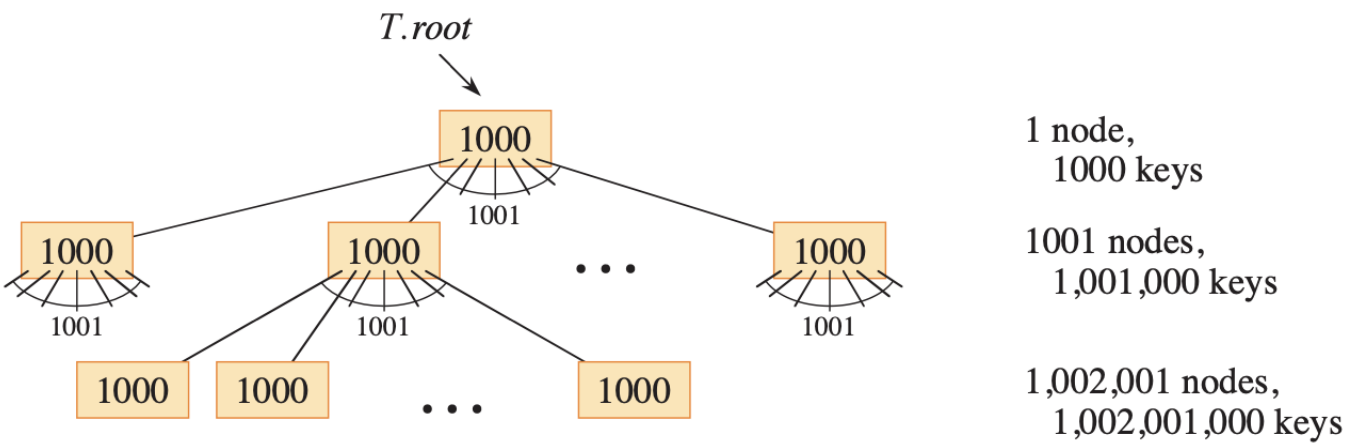


Figure: A B-tree of height 2 containing over one billion keys.

Comparison with BST

- A Binary Search Tree (BST) has only one key per node, making it deep, requiring many disk reads.
- A B-tree node contains many keys, making the tree shallow, requiring fewer disk reads.

✅ **Fewer levels in the tree → Fewer disk accesses → Faster search times.**

- Searching in a B-Tree vs. Binary Search Tree

Tree Type	Depth (Height)	Disk Reads Required
Binary Search Tree (1 key per node)	**Height ≈ $\log_2(N)$**	Many disk reads
B-Tree (1000 keys per node)	**Height ≈ $\log_{1000}(N)$**	Fewer disk reads

A simple B-tree

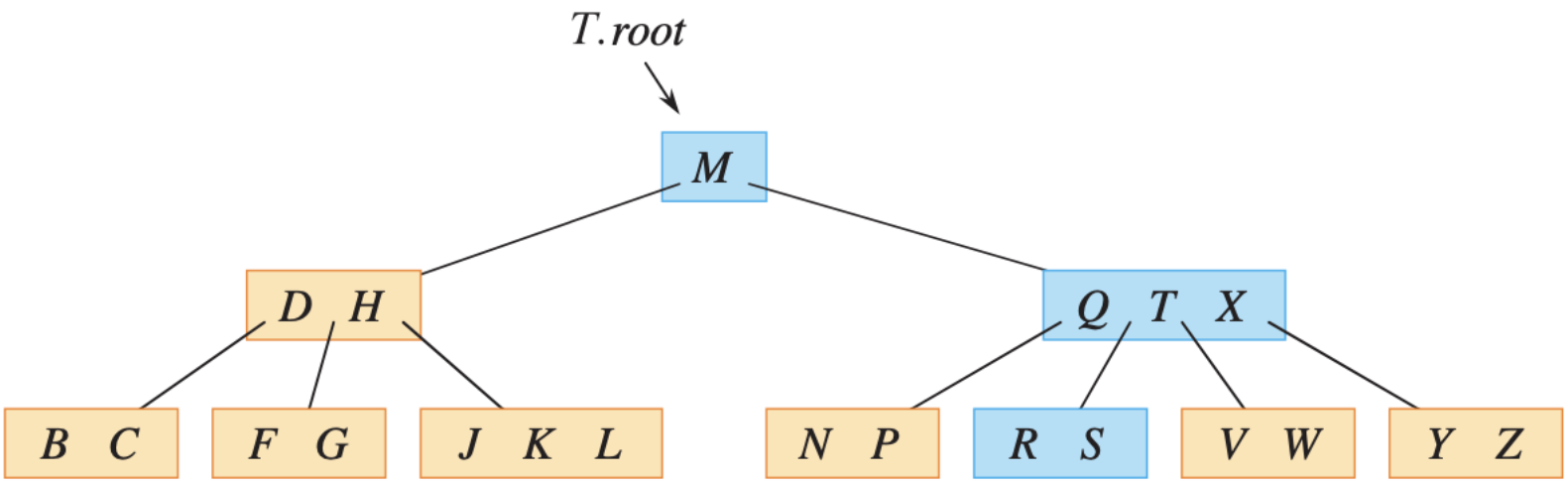


Figure: A B-tree whose keys are the consonants of English.

B-Trees

- B-trees have *five structural properties* that ensure efficient operations and maintain balance.
- These properties help B-trees achieve $O(\log n)$ performance for insertions, deletions, and searches by maintaining an orderly and balanced structure.
- This efficiency makes them particularly suitable for systems where large blocks of data are managed and accessed frequently.

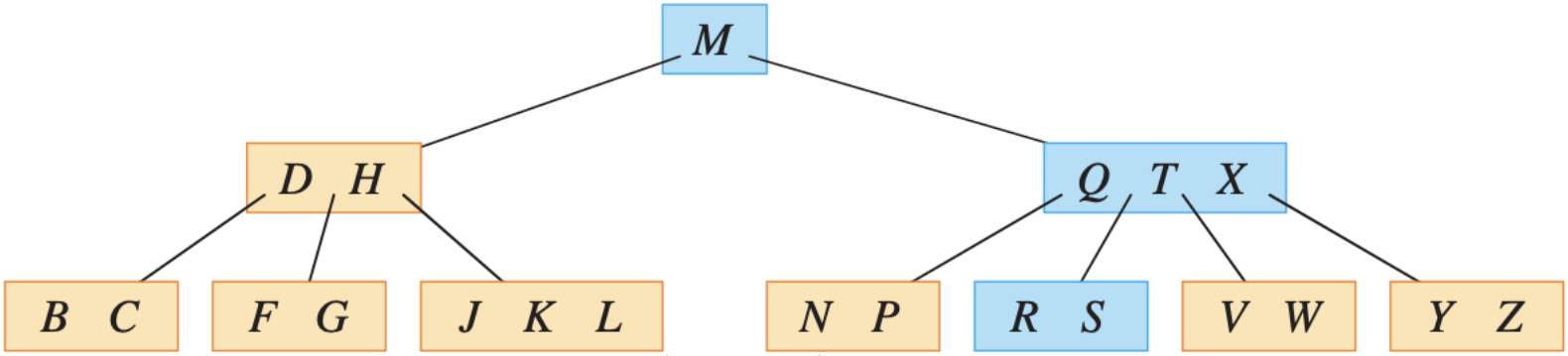
Five Structural Properties

1. Every node x has the following attributes:

a. $x.n$, the number of keys currently stored in node x,

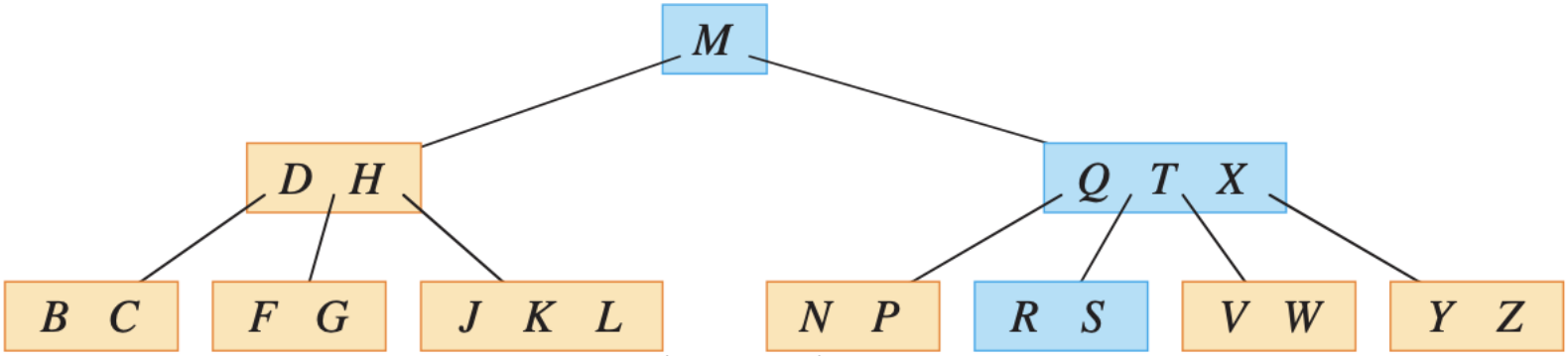
b. the $x.n$ keys themselves, $x.key_1, x.key_2, \ldots, x.key_{x.n}$, stored in monotonically increasing order, so that $x.key_1 \leq x.key_2 \leq \ldots \leq x.key_{x.n}$,

c. $x.leaf$, a boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.



Five Structural Properties

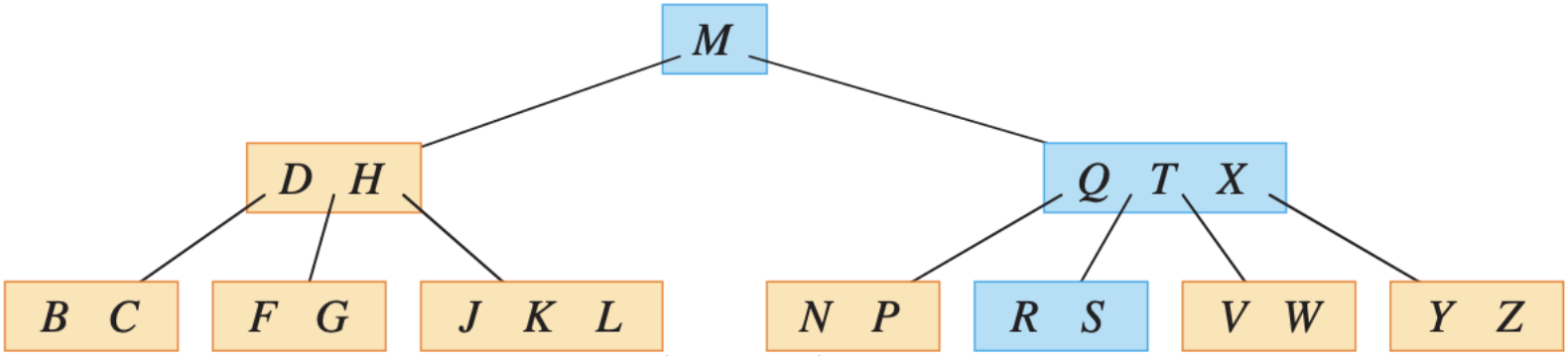
2. Each internal node x also contains $x.n + 1$ pointers $x.c_1, x.c_2, \ldots, x.c_{x.n+1}$ to its children. Leaf nodes have no children, and so their c_i attributes are undefined.



Five Structural Properties

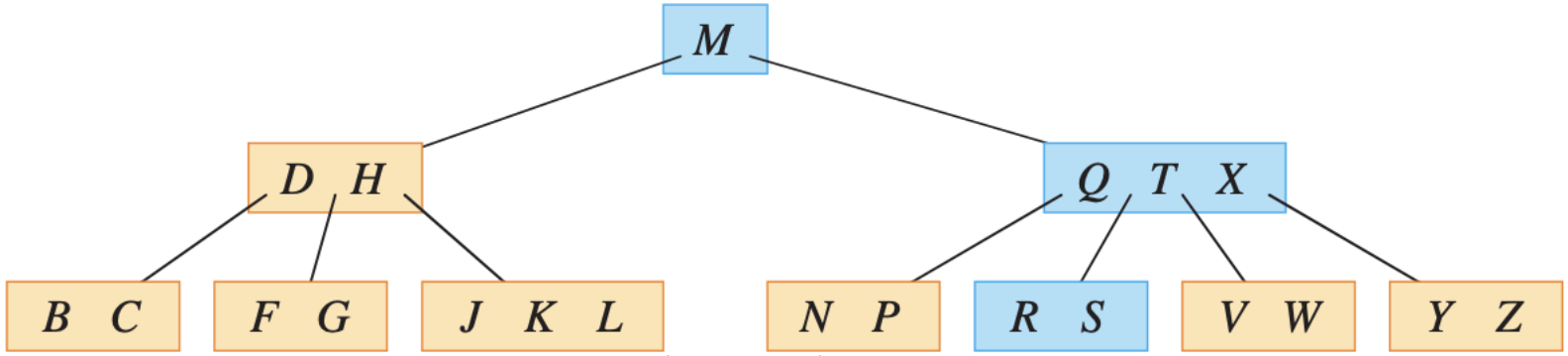
3. The keys $x.\text{key}_i$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root $x.c_i$, then

$$k_1 \leq x.\text{key}_1 \leq k_2 \leq x.\text{key}_2 \leq \ldots \leq x.\text{key}_{x.n} \leq k_{x.n+1}$$



Five Structural Properties

4. All leaves have the same depth, which is the tree's height h.



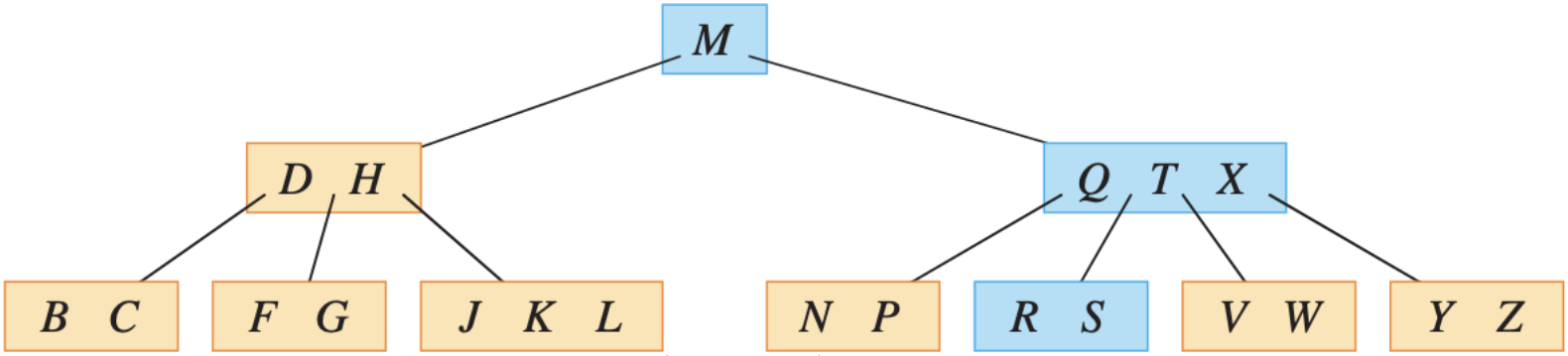
Five Structural Properties

5. Nodes have lower and upper bounds on the number of keys they can contain, expressed in terms of a fixed integer $t \geq 2$ called the ***minimum degree*** of the B-tree:

   a. Every node other than the root must have at least $t - 1$ keys. Every internal node other than the root thus has at least t children. If the tree is nonempty, the root must have at least one key.

   b. Every node may contain at most $2t - 1$ keys. Therefore, an internal node may have at most $2t$ children. We say that a node is ***full*** if it contains exactly $2t - 1$ keys.[3]

The simplest B-tree occurs when $t = 2$. Every internal node then has either 2, 3, or 4 children, and it is a ***2-3-4 tree***. In practice, however, much larger values of t yield B-trees with smaller height.

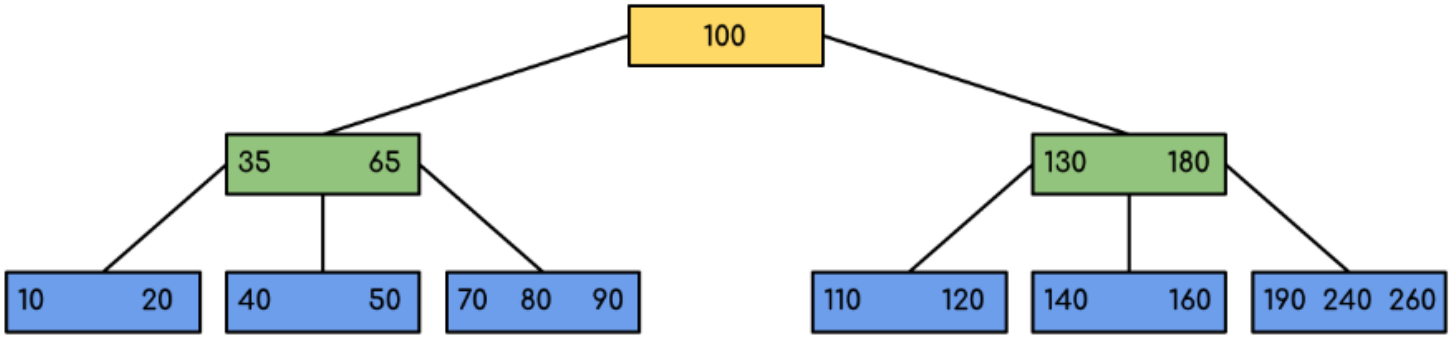


Search Operation

Search

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

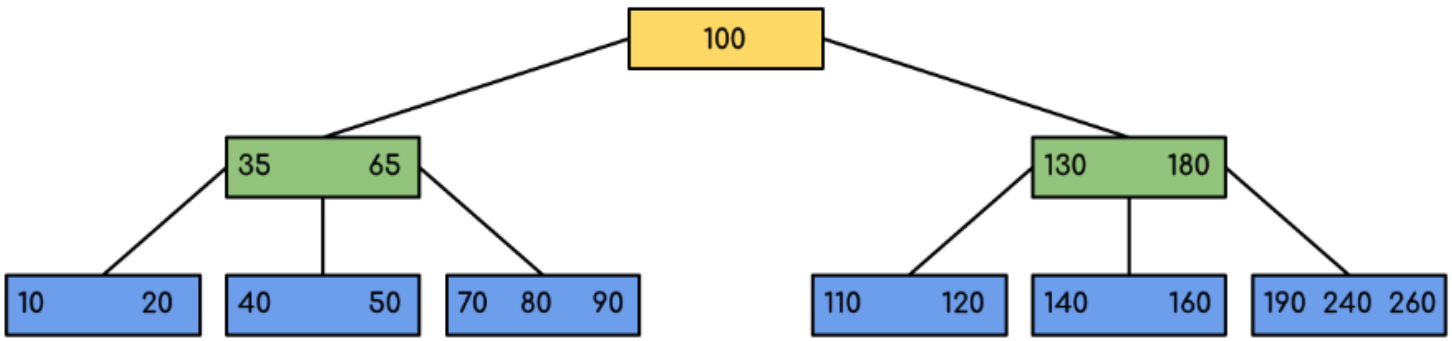
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Search

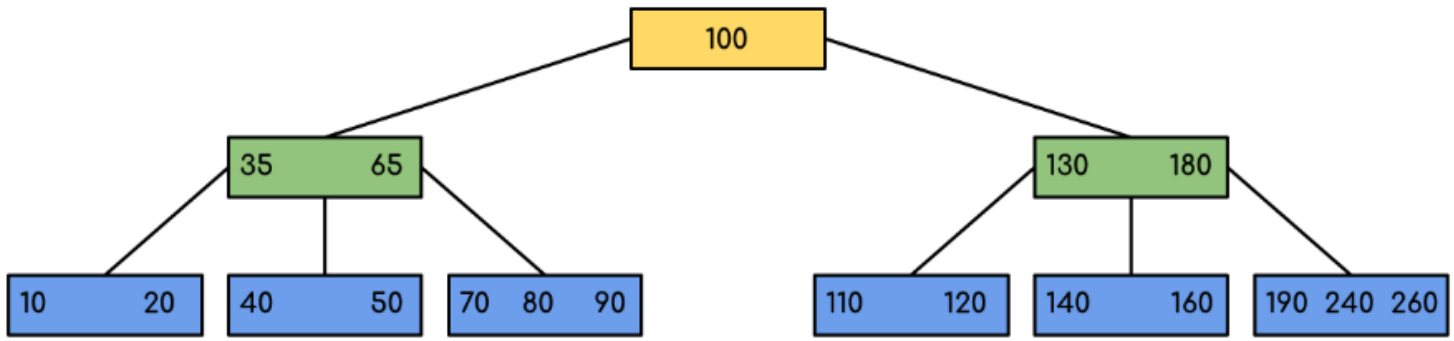


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x = [100]$ and $k = 120$.

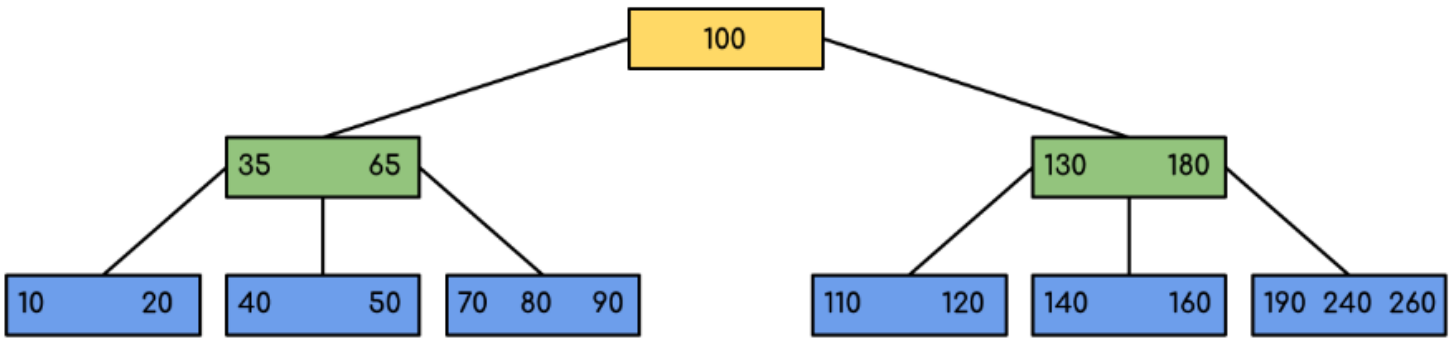
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

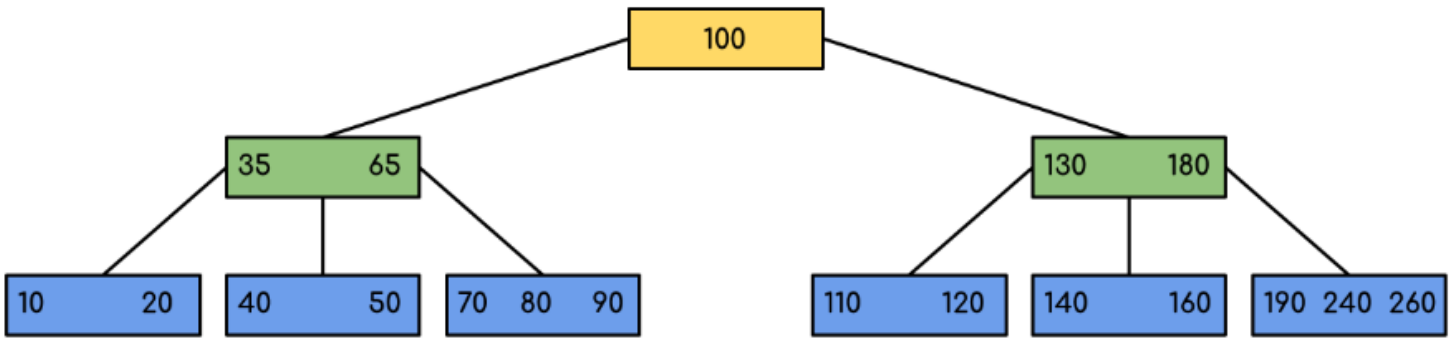
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Search

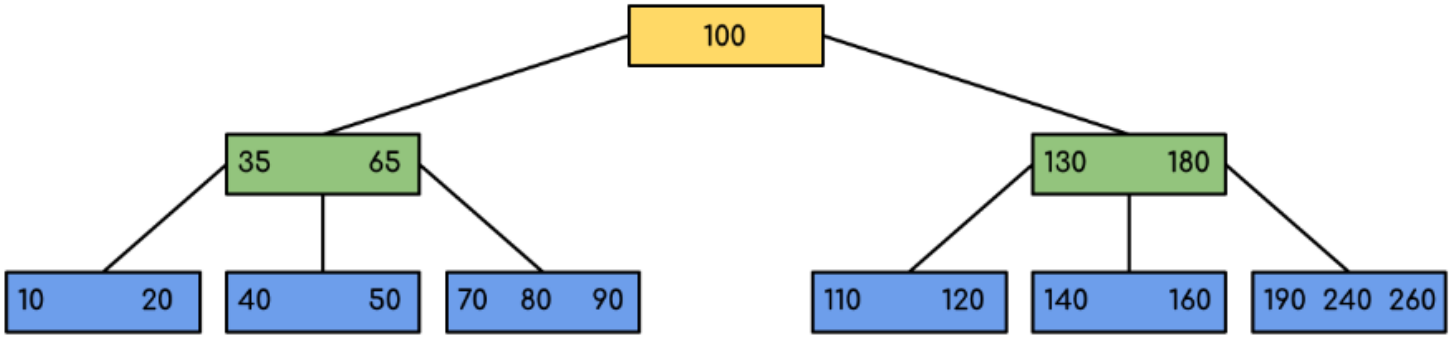


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$i = 2.$

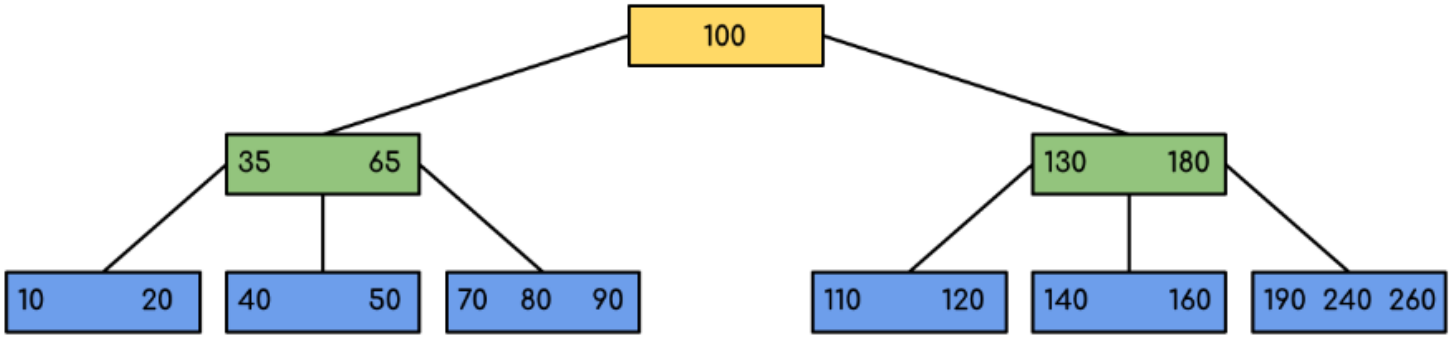
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

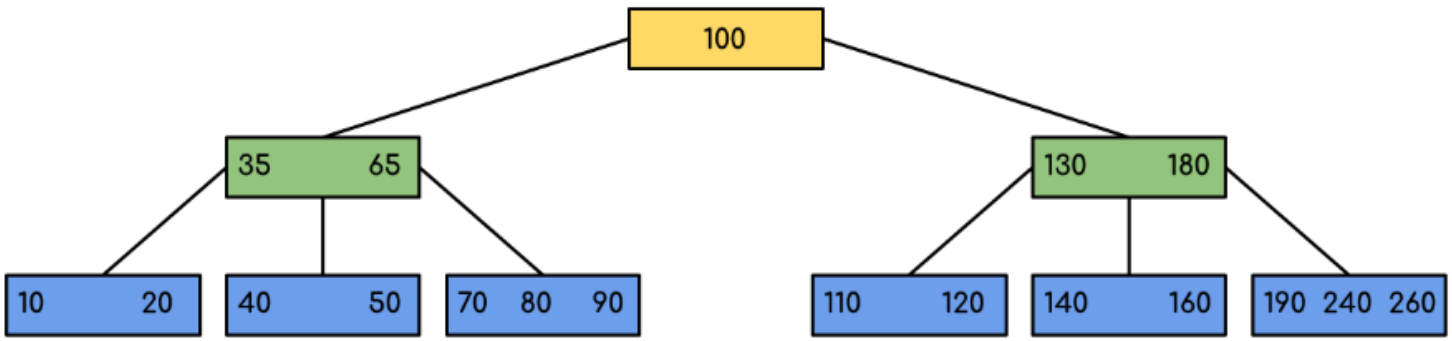
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Search

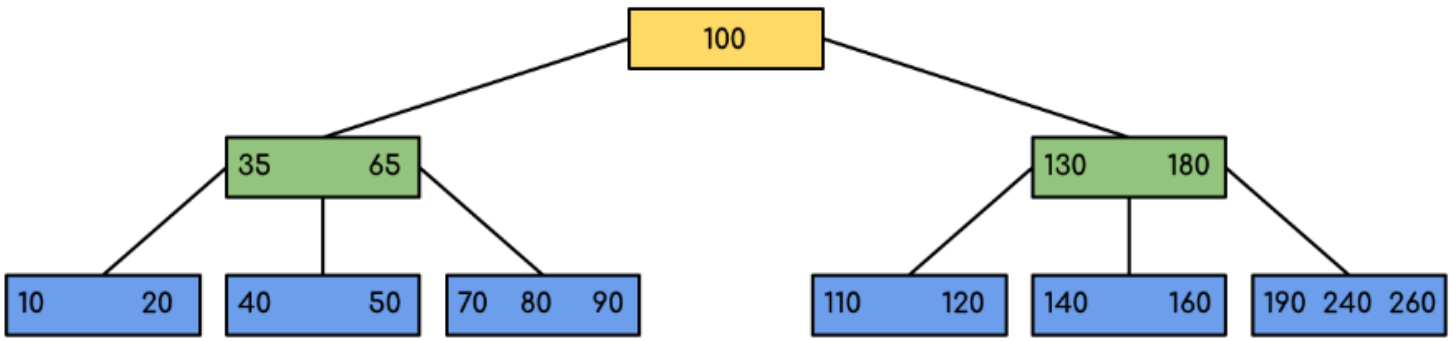


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x.c[2] = [130, 180]$.

Search

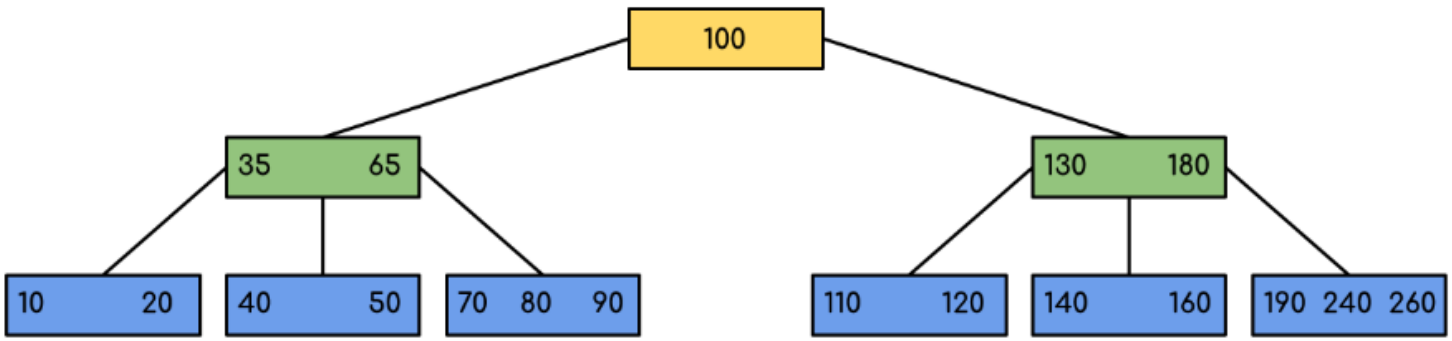


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x.c[2] = [130, 180]$ and $k = 120$.

Search

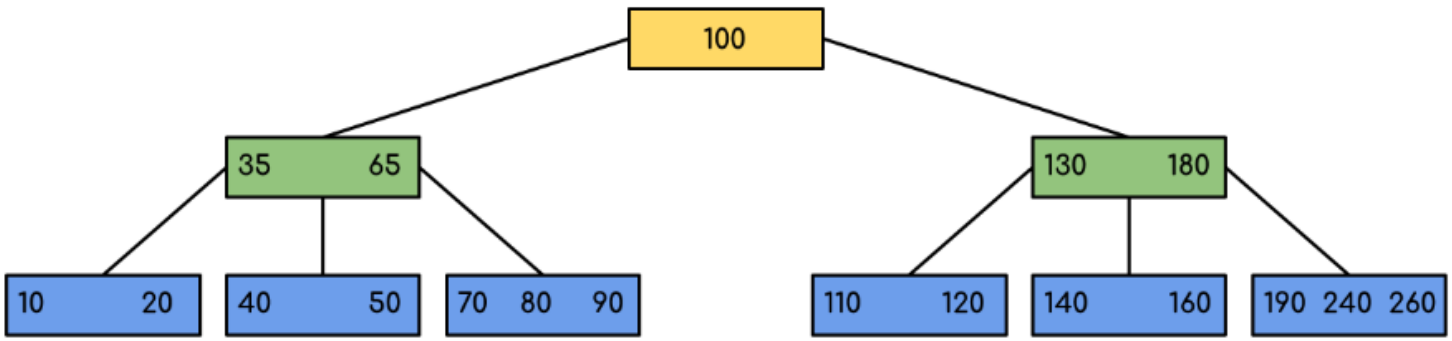


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x = [130, 180]$ and $k = 120$.

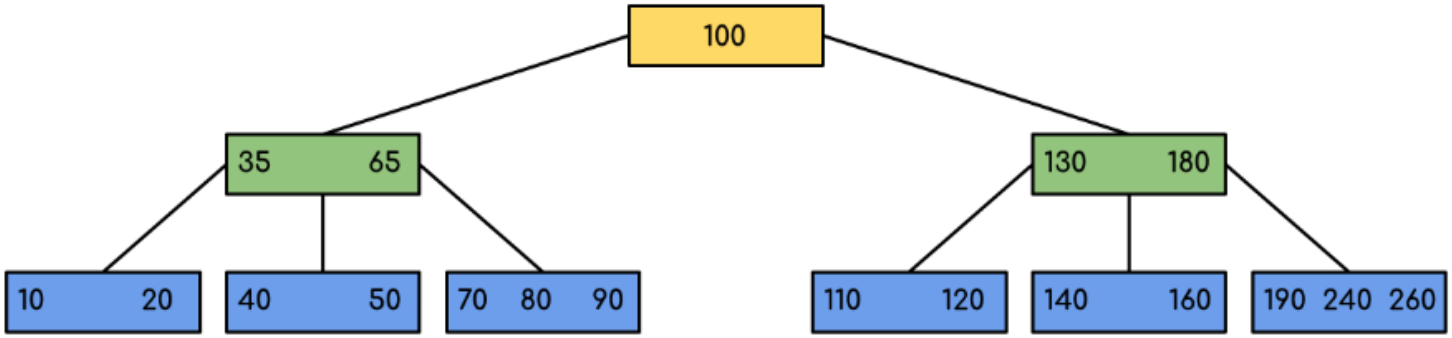
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

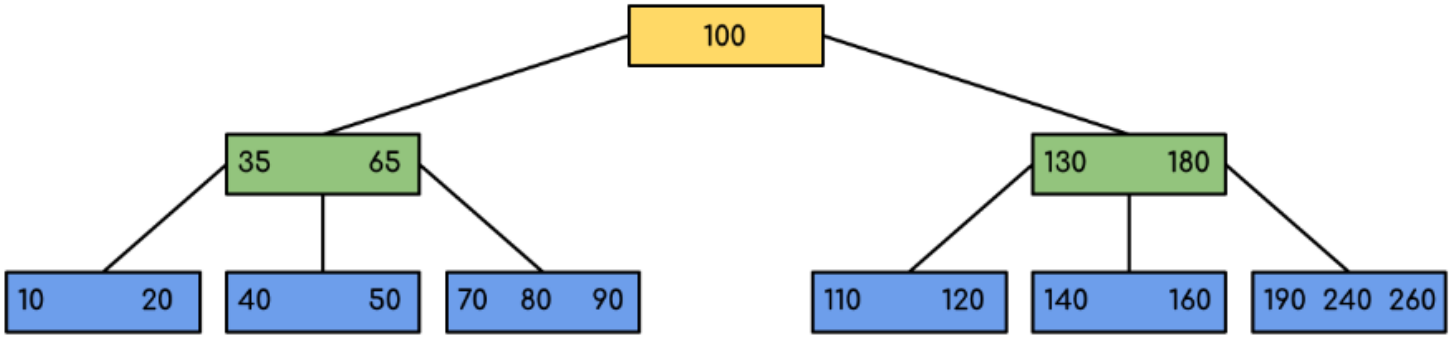
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

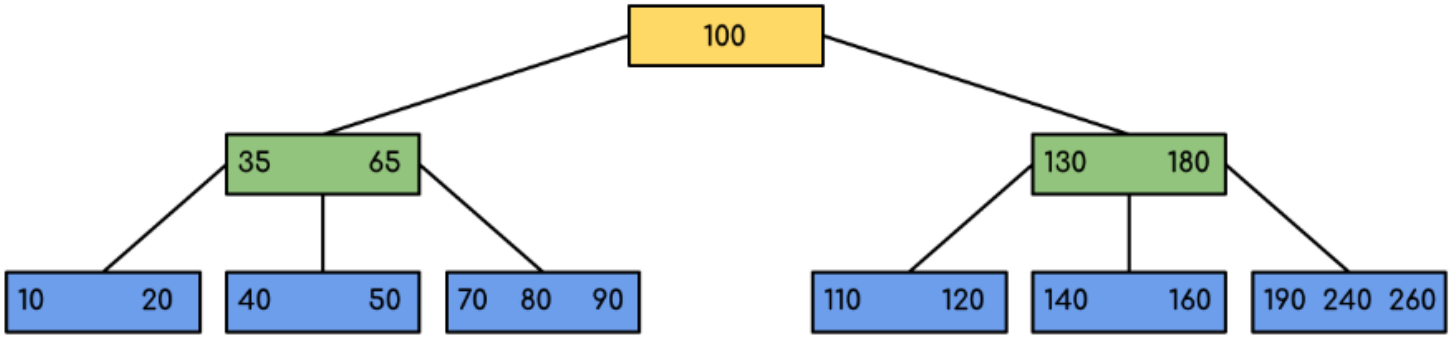
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

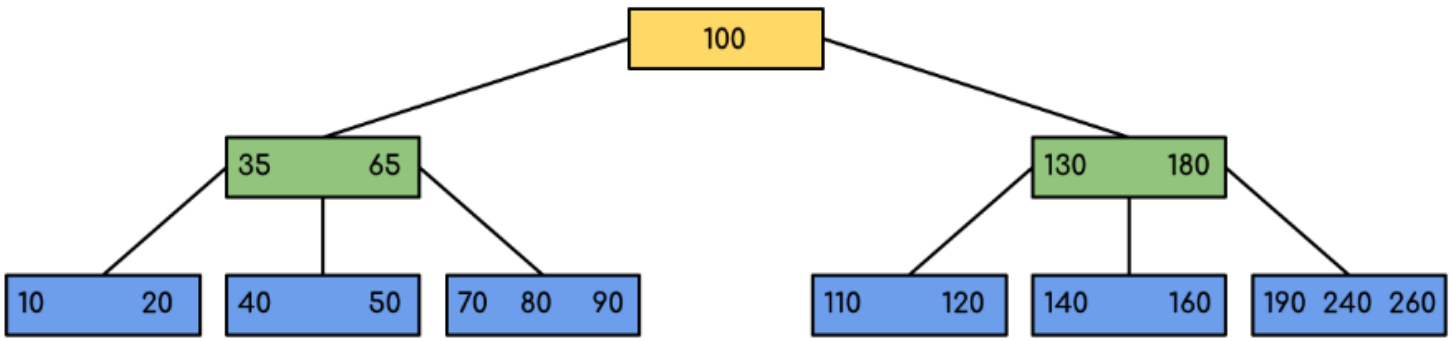
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Search

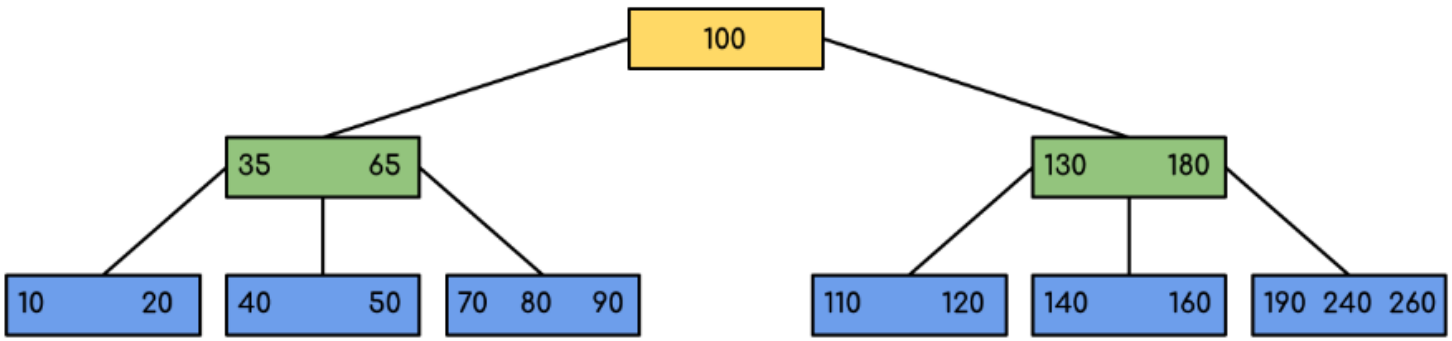


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x.c[1] = [110, 120].$

Search

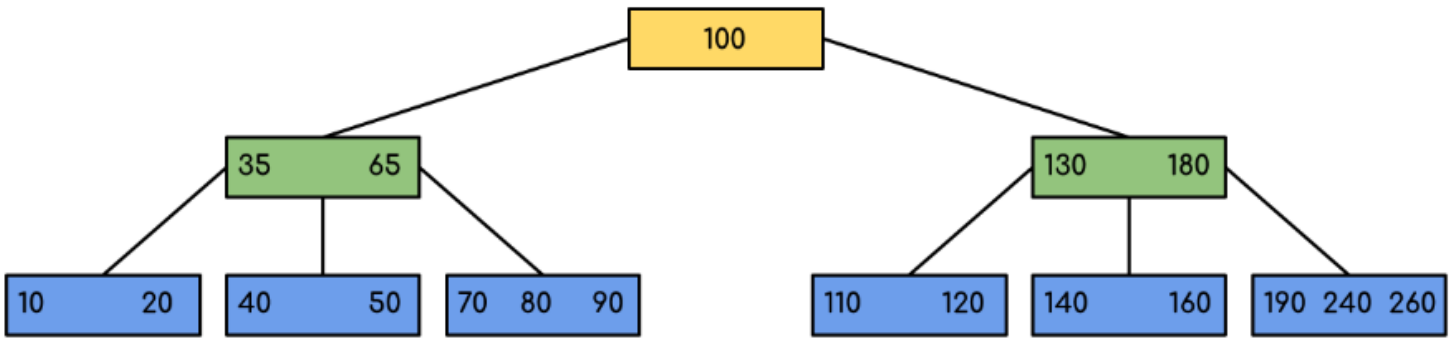


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x.c[1] = [110, 120]$ and $k = 120$.

Search

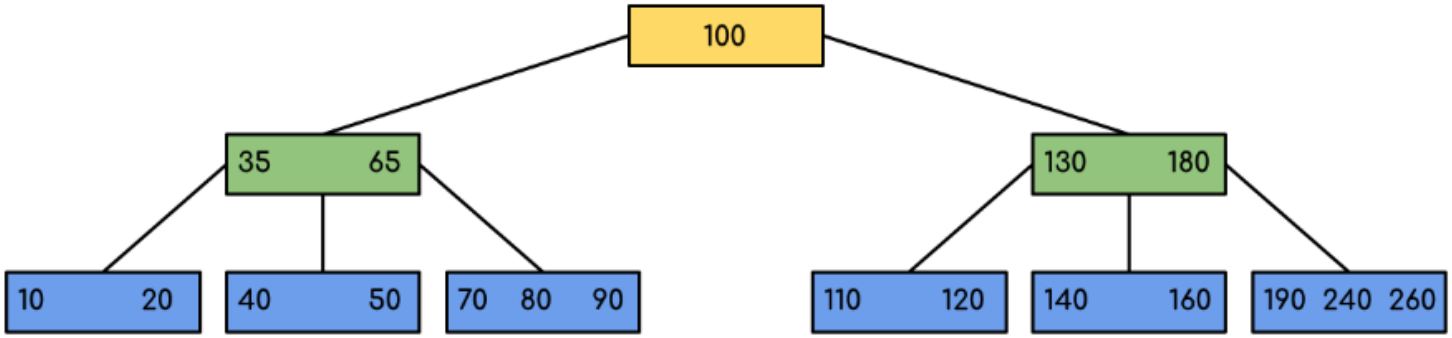


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x = [110, 120]$ and $k = 120$.

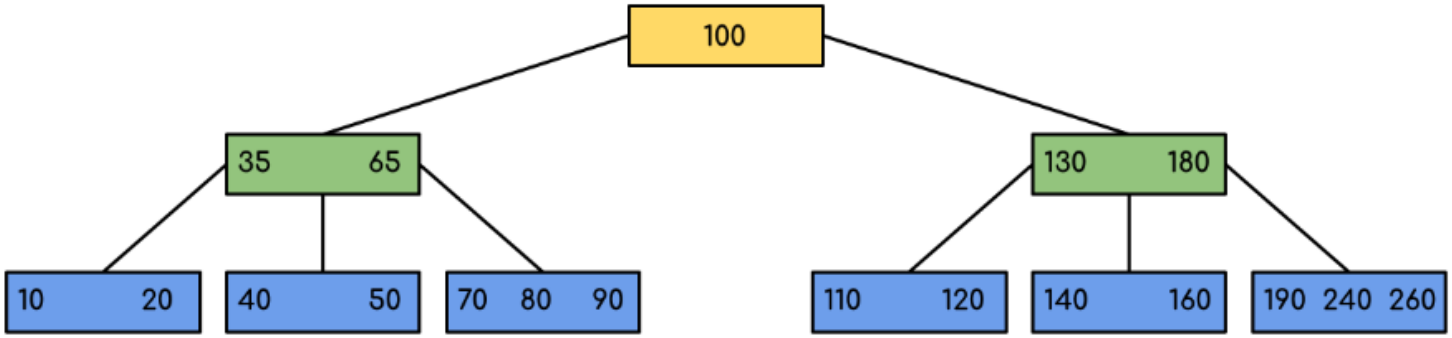
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

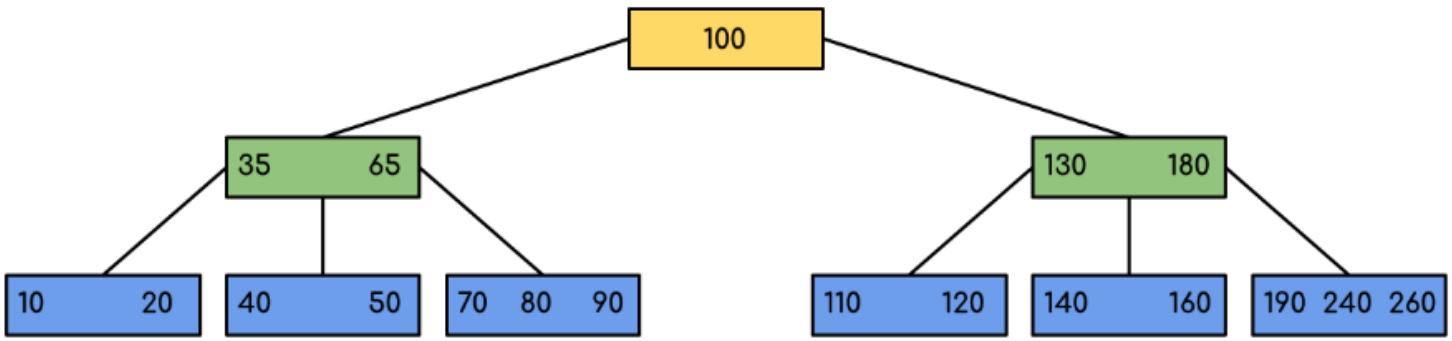
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Search

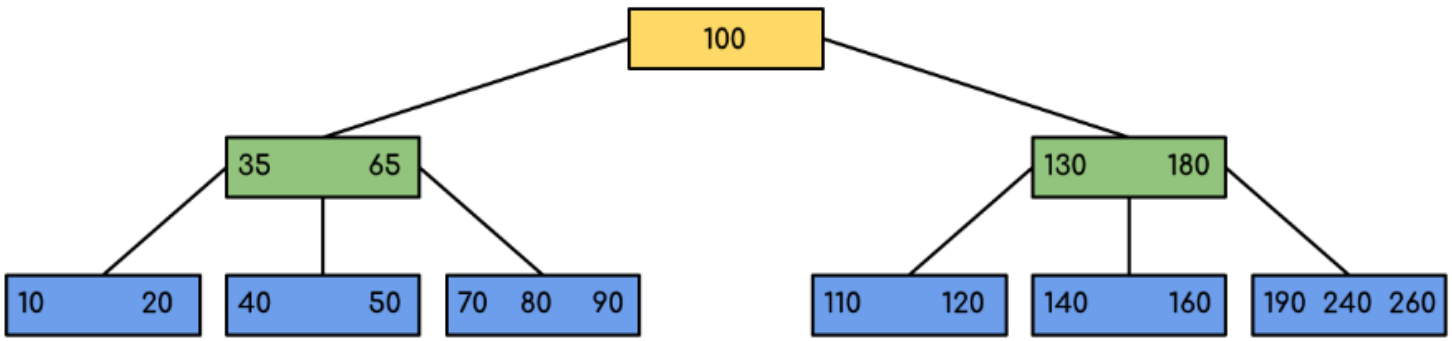


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$i = 2.$

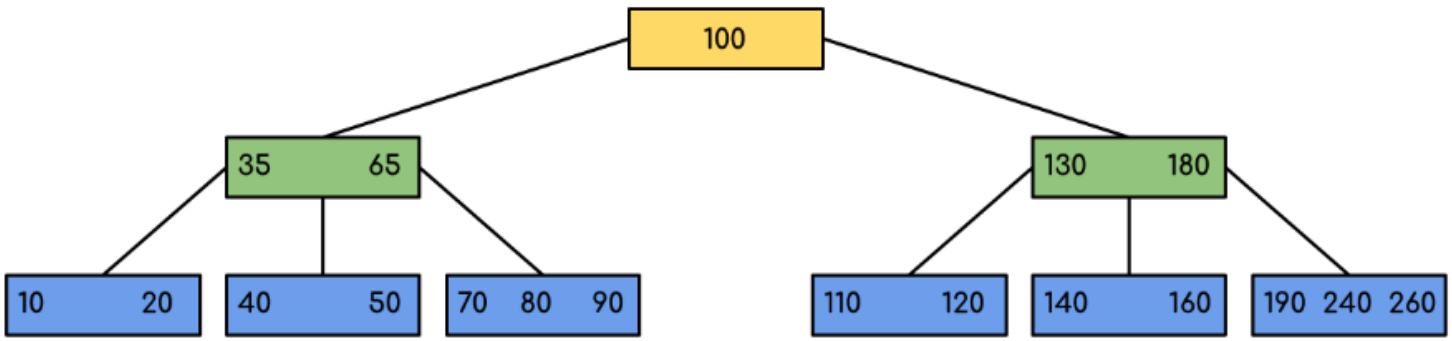
Search



$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Search

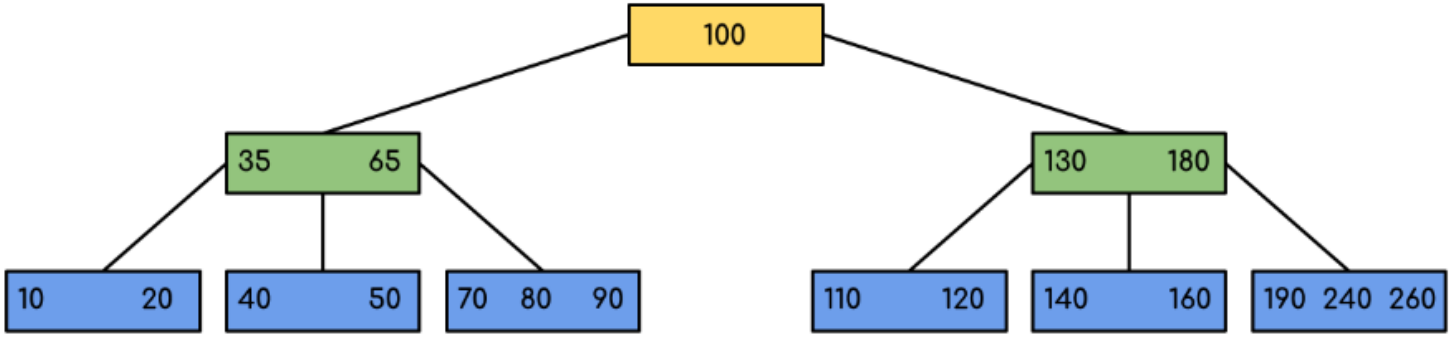


$k = 120$

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

$x = [110, 120]$ and $i = 2$.

Search




```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]   *Where should key be located in this node?
        i = i + 1
    if i <= x.n and k == x.key[i]     *Can we find the key in the current node?
        return (x, i)
    elseif x.leaf                     *If not, are we at a leaf node?
        return NIL
    else DISK-READ(x.c[i])            *If not, which child should we search next?
    return B-TREE-SEARCH(x.c[i], k)
```

DISK-READ()

- The *DISK-READ* operation is a function that deals with reading data from a secondary storage device, like a hard disk, into main memory.
- The *primary purpose* is to load a specific node of the B-tree from disk into main memory for processing. In a B-tree used in a database, the nodes are typically stored on disk, and DISK-READ is necessary to interact with these nodes during search, insert, or delete operations.
- In the context of B-trees, which are often used in databases and file systems, this operation is crucial because B-trees can be very large and not entirely fit into main memory.

Usage in B-Trees

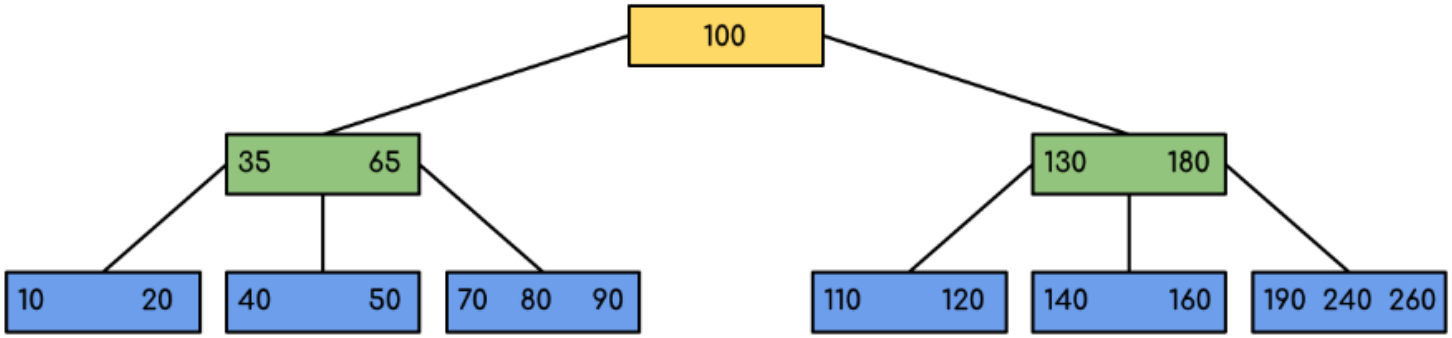
- In the *B-TREE-SEARCH* algorithm, when the search for a key descends from the current node to one of its children, the child node may not be in main memory. The *DISK-READ* operation is used to load the child node from disk into main memory so that the search can continue.
- Typically, *DISK-READ* takes the address or identifier of the node (often referred to as a pointer or a disk block address) as a parameter. In the pseudocode, $x.c_i$ would be the identifier for the child node i of node x, where c_i points to the location on the disk where this child node is stored.

** It ensures that the data is brought into main memory.

Example

- When a B-tree node is accessed during a search and it is determined that the desired key might be in a child node, the program will execute DISK-READ($x.c_i$) to read the i-th child from disk into memory.
- Once the node is in memory, the search function B-TREE-SEARCH can be recursively called on this node to continue the search.

Search




```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.key[i]
        i = i + 1
    if i <= x.n and k == x.key[i]
        return (x, i)
    elseif x.leaf
        return NIL
    else DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)
```

Inserting Operation

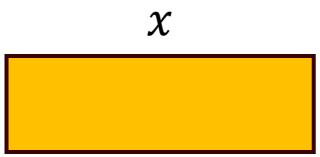
Insertion Operation

```
B-TREE-CREATE(T)
B-TREE-INSERT(T, k)
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

```
B-TREE-SPLIT-ROOT(T)
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Creating an empty B-tree

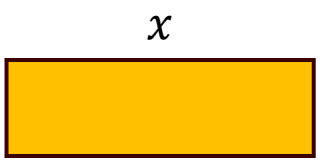
```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```



Creating an empty B-tree

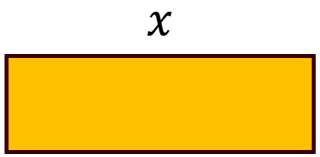
```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```

in memory



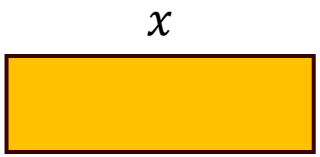
Creating an empty B-tree

```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```



Creating an empty B-tree

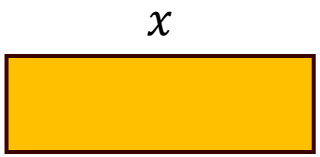
```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```



Creating an empty B-tree

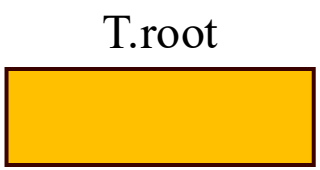
```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```

Save it to disk storage



Creating an empty B-tree

```
B-TREE-CREATE(T)
    x = ALLOCATE-NODE()
    x.leaf = TRUE
    x.n = 0
    DISK-WRITE(x)
    T.root = x
```



Inserting a key into a B-tree

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

Inserting 5 into a B-tree

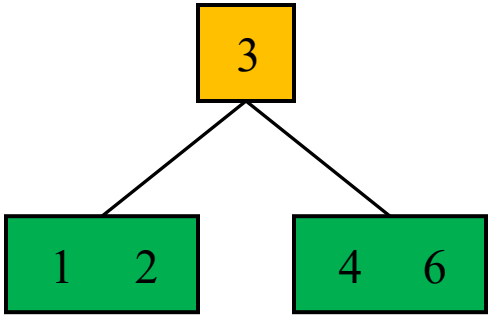
***Focusing on B-Tree-Insert-Nonfull(x, k).

Inserting 5 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

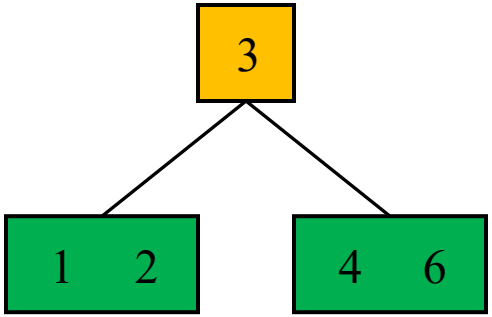


Inserting 5 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

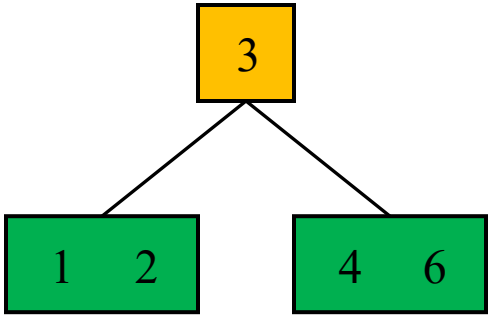


Inserting 5 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```



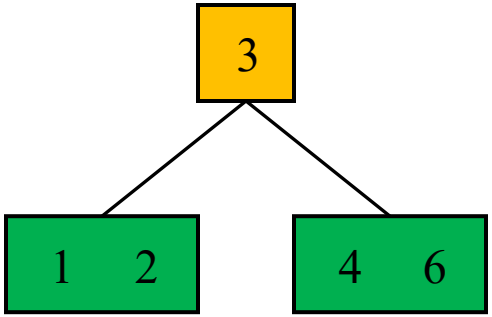
Inserting 5 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

- $r = [3]$.
- $k = 5$.



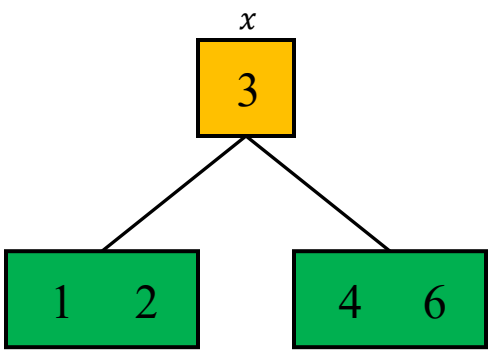
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

r = [3] and k = 5.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



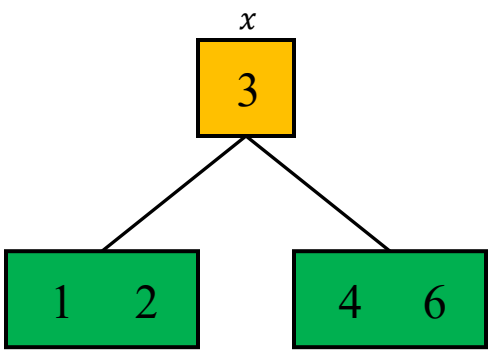
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$i = 1.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

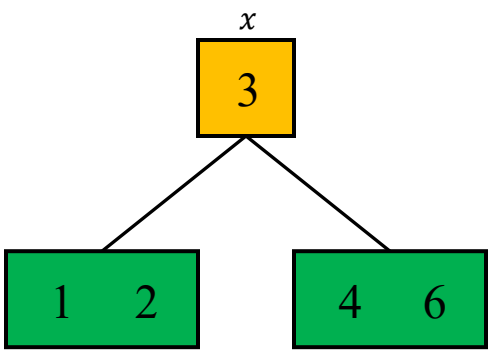


Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

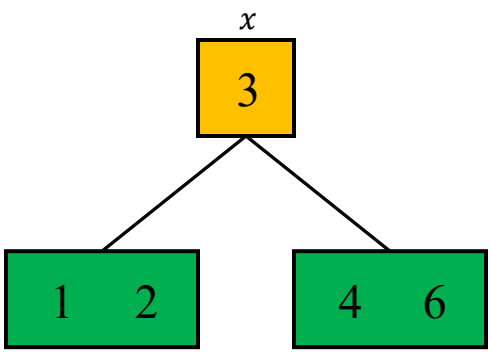


Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



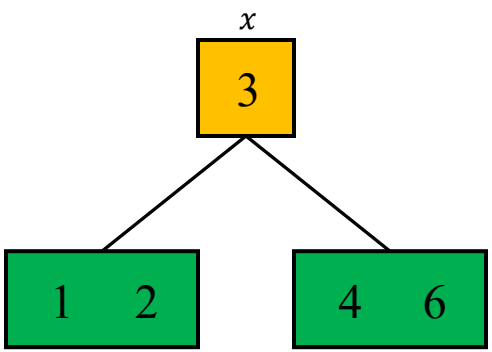
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$i = 1.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

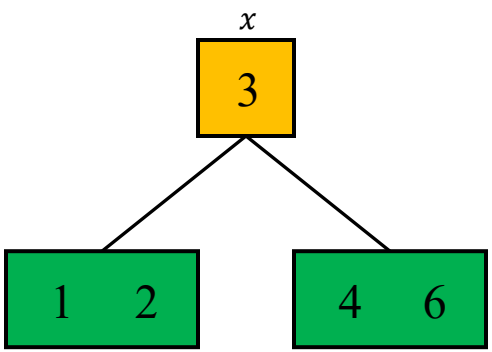


Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1 ←——— i = 2.
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



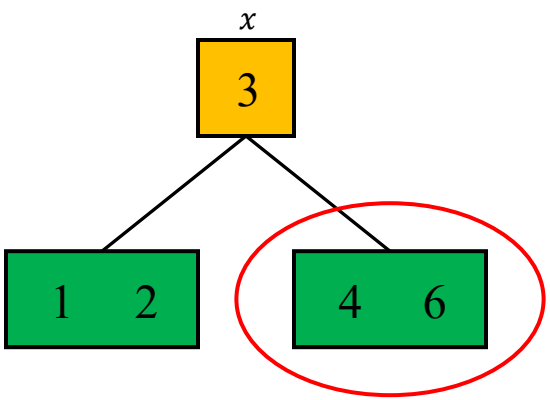
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.c_2 = [4, 6]$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

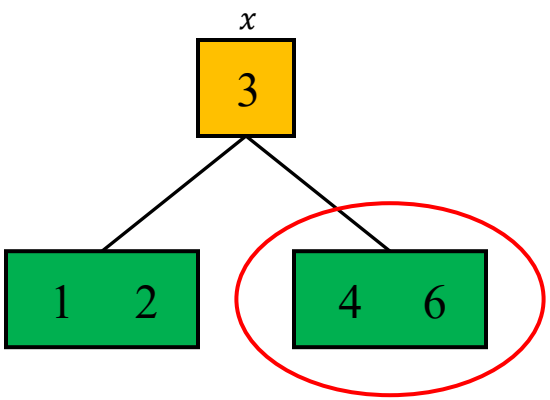


Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1        ← 2 == 3 ???
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



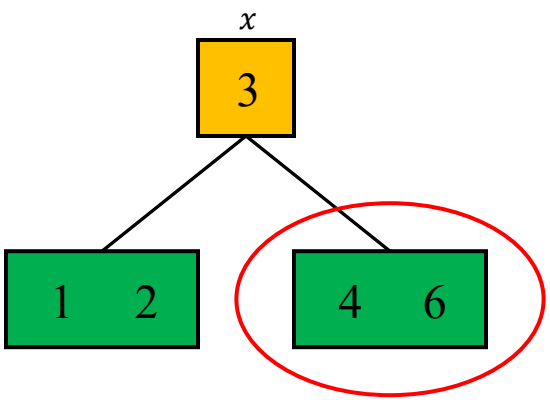
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.c_2 = [4, 6]$ and $k = 5$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



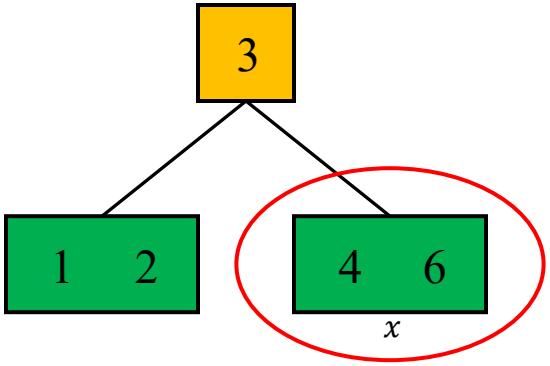
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.c_2 = [4, 6]$ and $k = 5$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



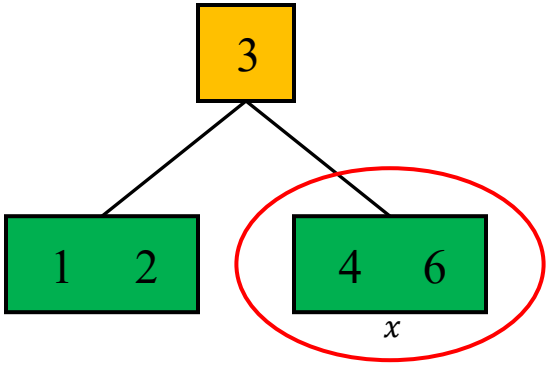
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.n = 2.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

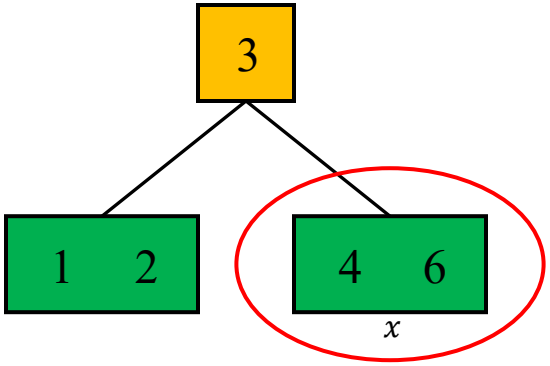


Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf ← True.
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



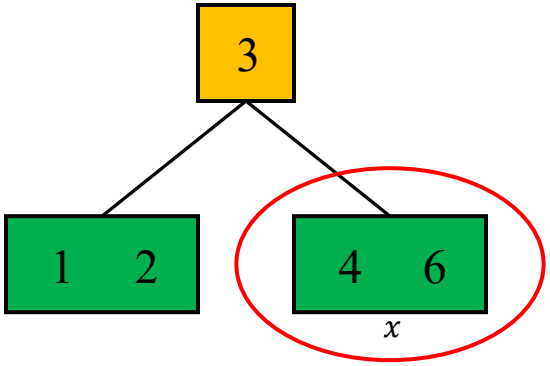
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

True.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



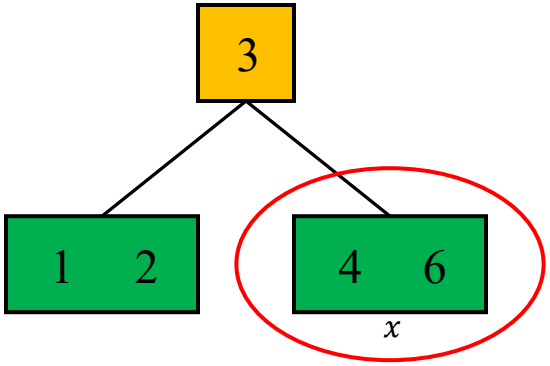
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.key_3 = x.key_2 = 6.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



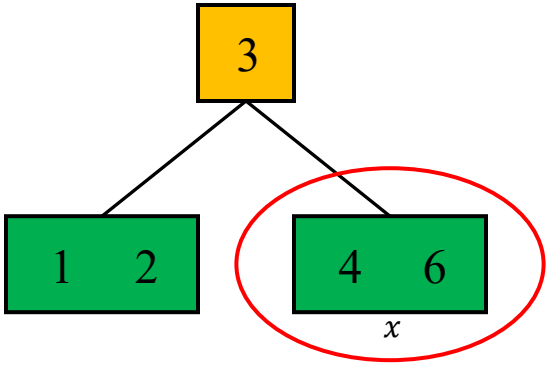
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$i = 1$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



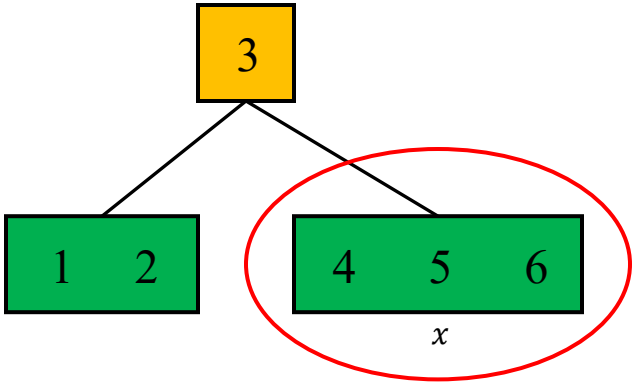
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.key_2 = 5$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$



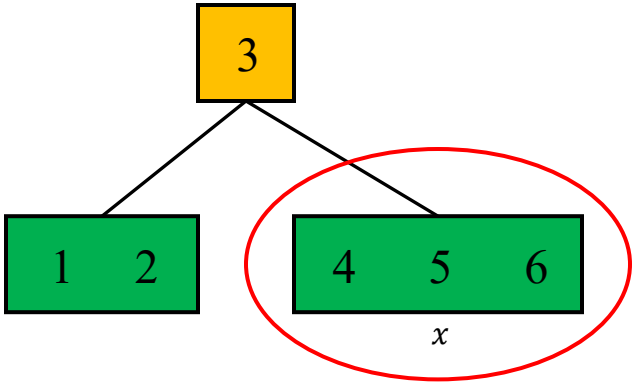
Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.n = 3$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$

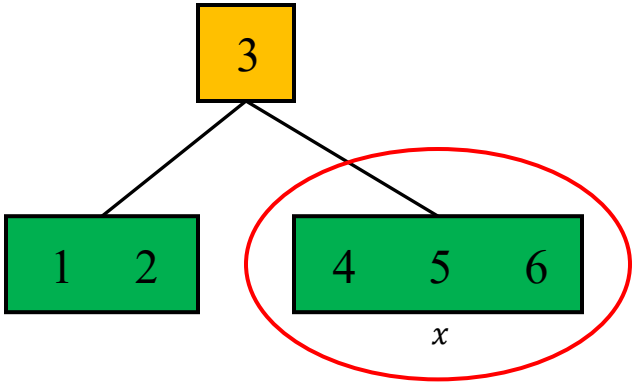


Inserting 5 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

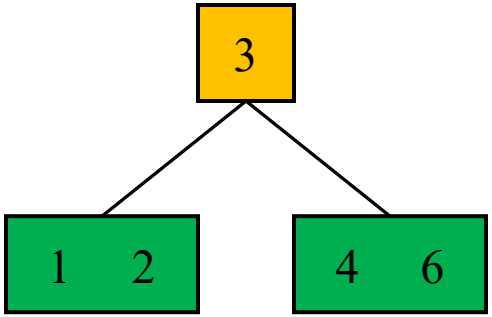
$k = 5$

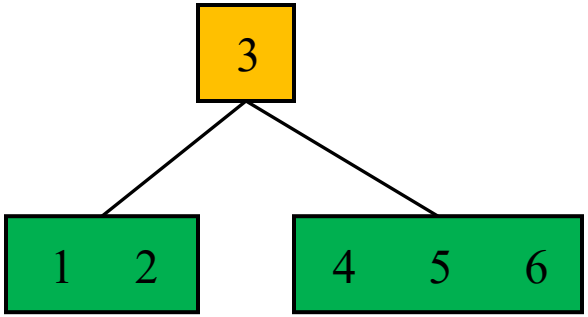


Inserting 5 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 5$





Inserting 7 into a B-tree

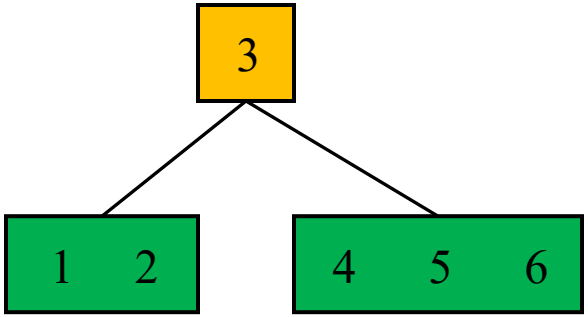
***Focusing on B-Tree-Split-Child(x, i).

Inserting 7 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

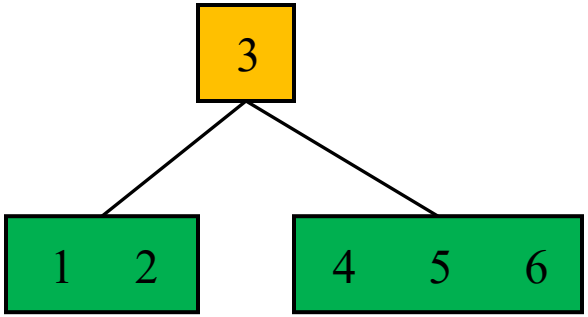


Inserting 7 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

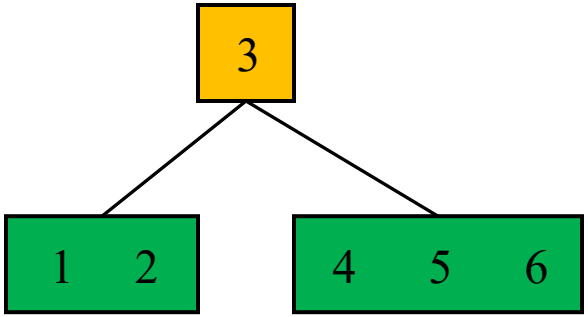


Inserting 7 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```



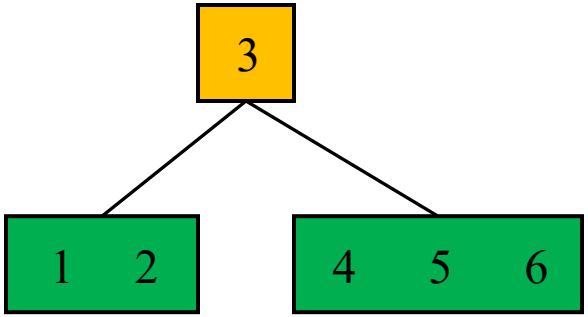
Inserting 7 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

- $r = [3]$
- $k = 7$



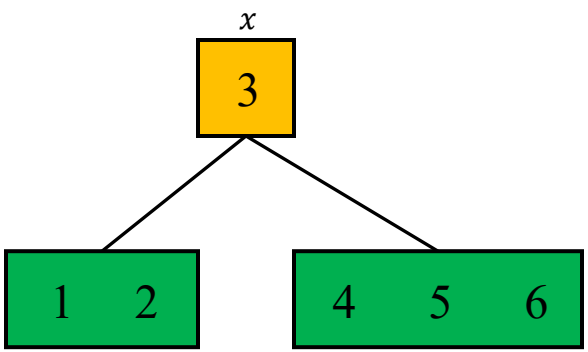
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x = [3]$ and $k = 7$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

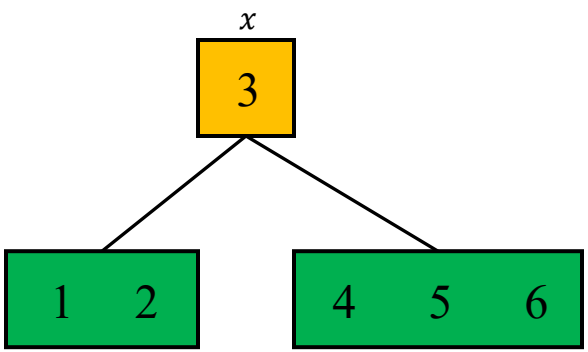


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n ← i = 1.
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

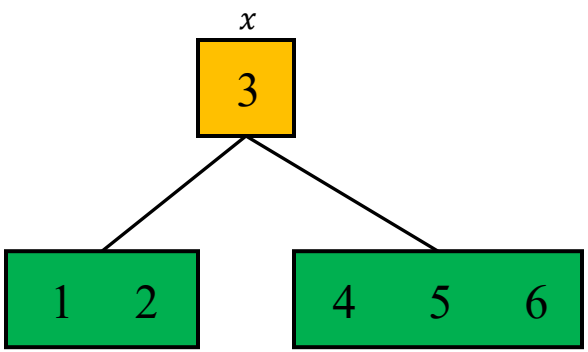


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

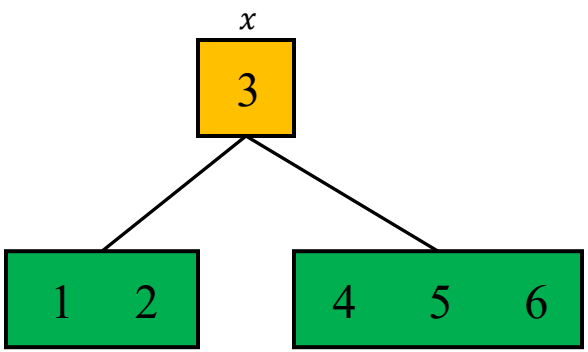


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

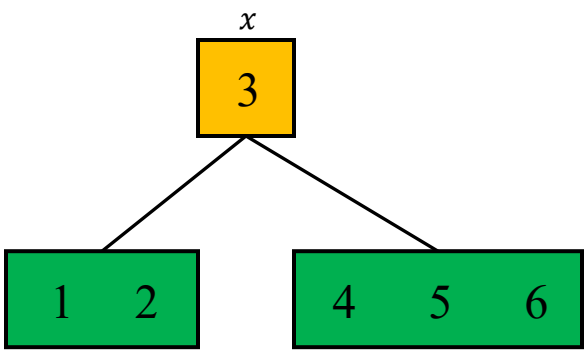


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

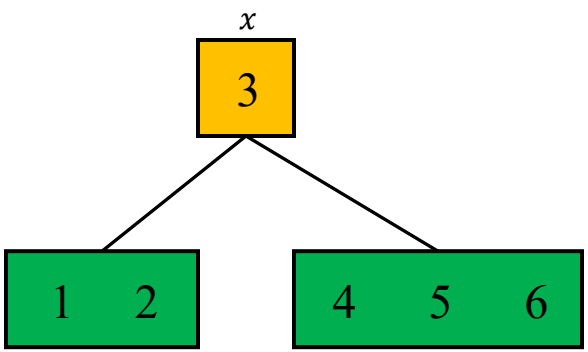


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1 ←——— i = 2.
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



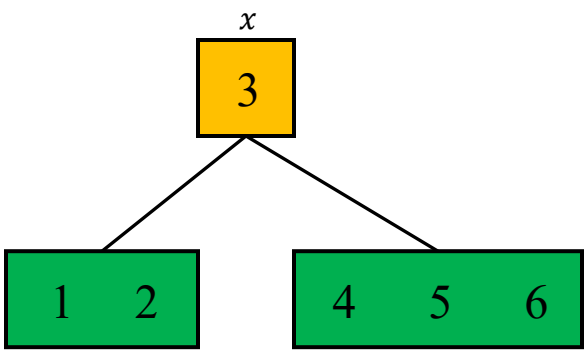
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.c_2 = [4, 5, 6]$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

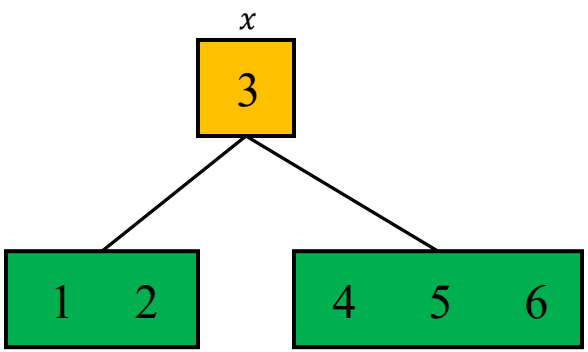


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1 ← True.
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



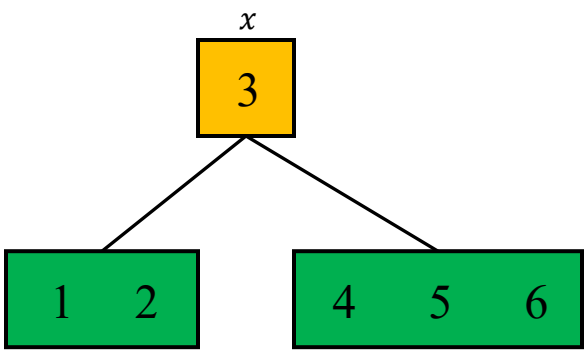
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x = [3]$ and $i = 2$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



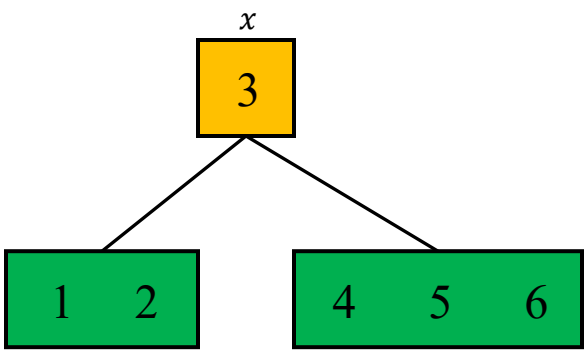
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$x = [3]$ and $i = 2$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



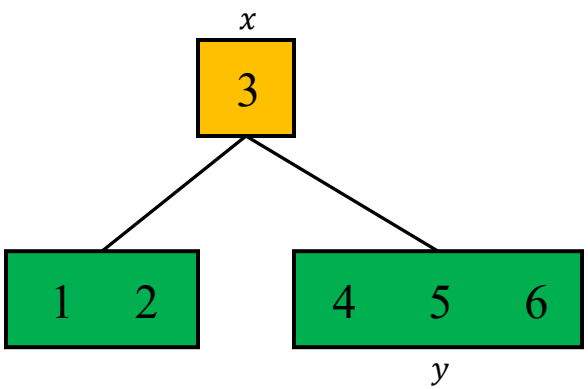
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$y = [4, 5, 6]$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

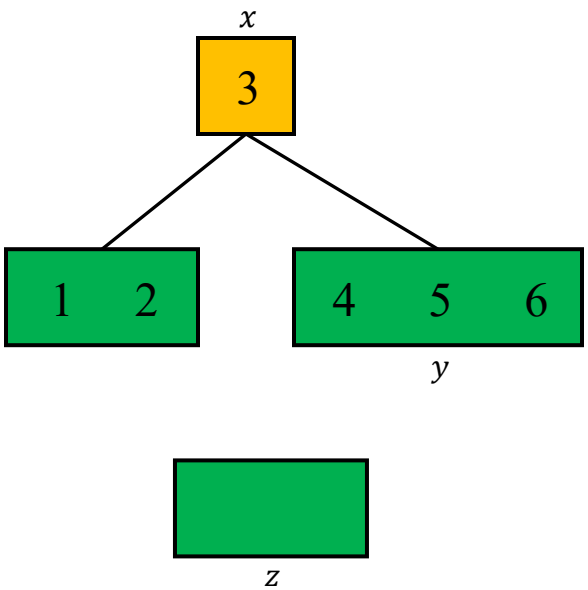


Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



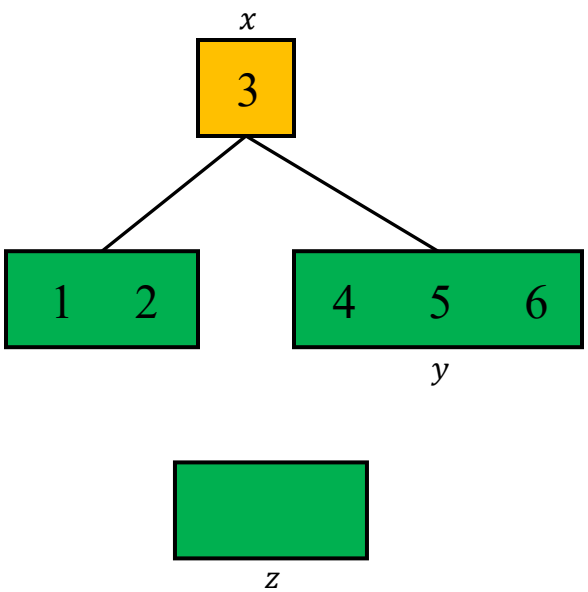
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

y is a leaf node.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



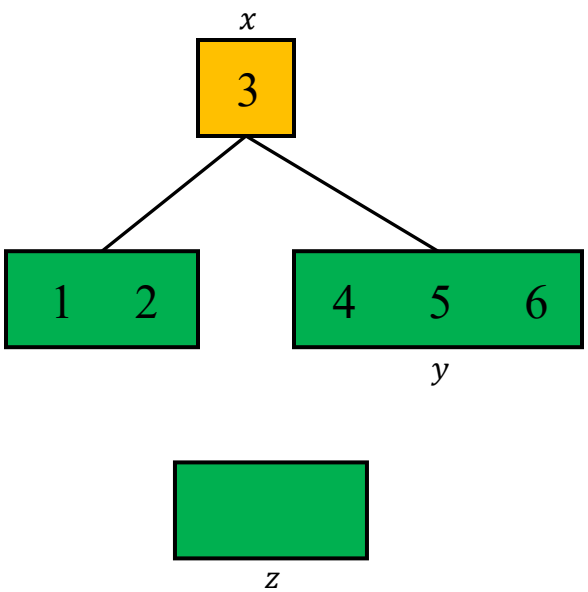
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

t is 2. → z.n = 1.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



$z.n = 1$

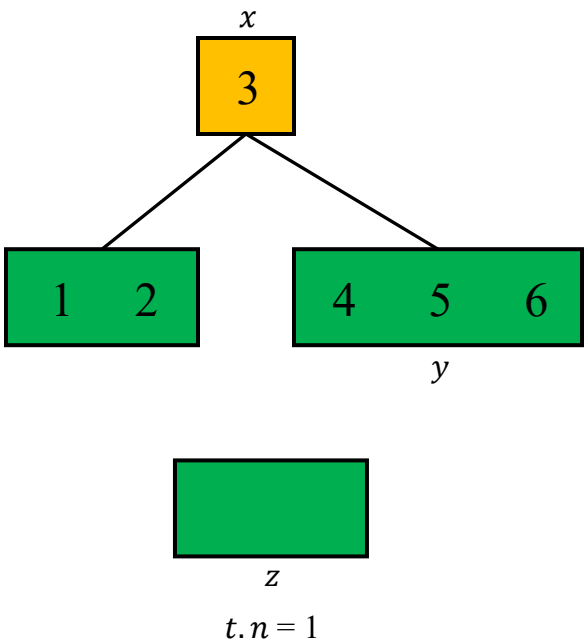
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Transfer of keys to z.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

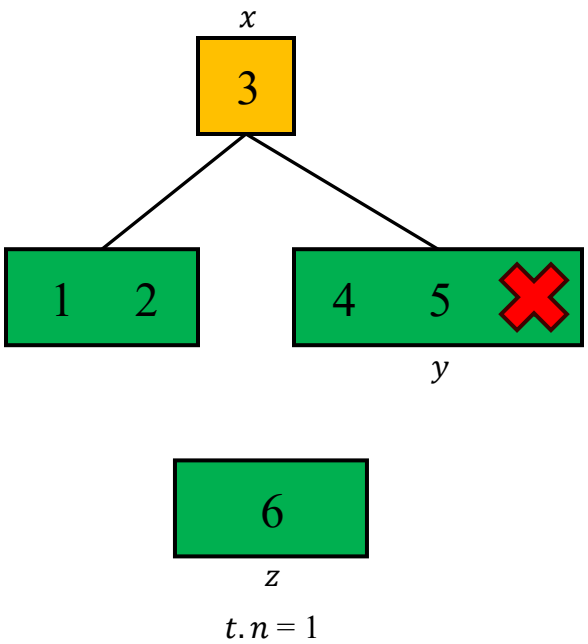


Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



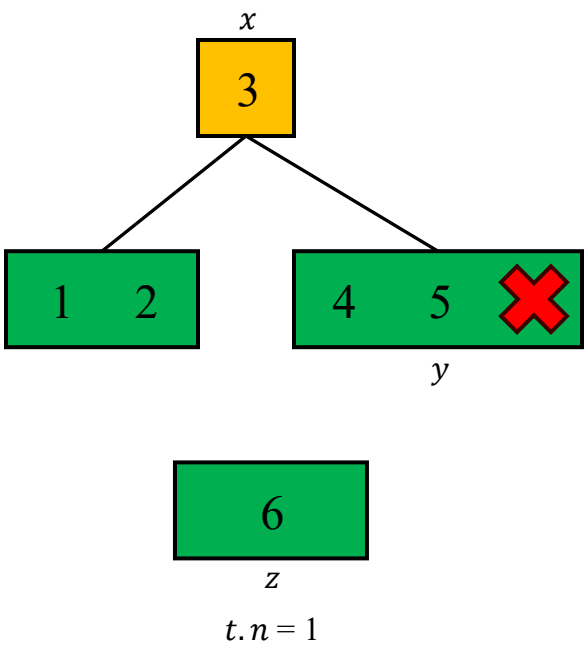
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

y is a leaf node.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



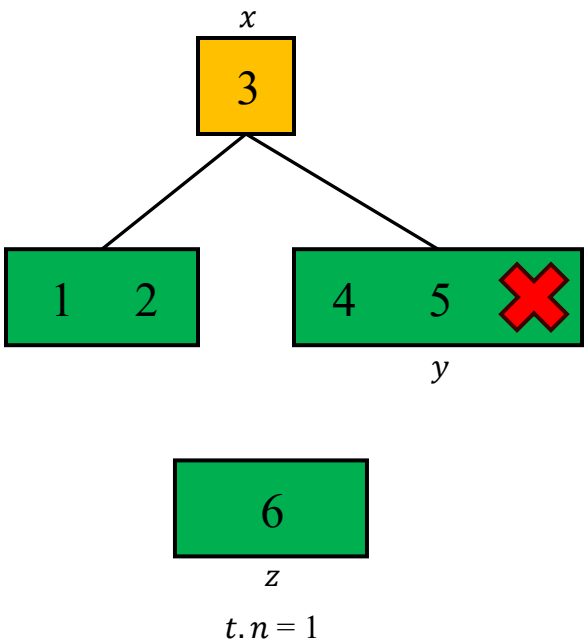
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Adjust y's key count.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



Inserting 7 into a B-tree

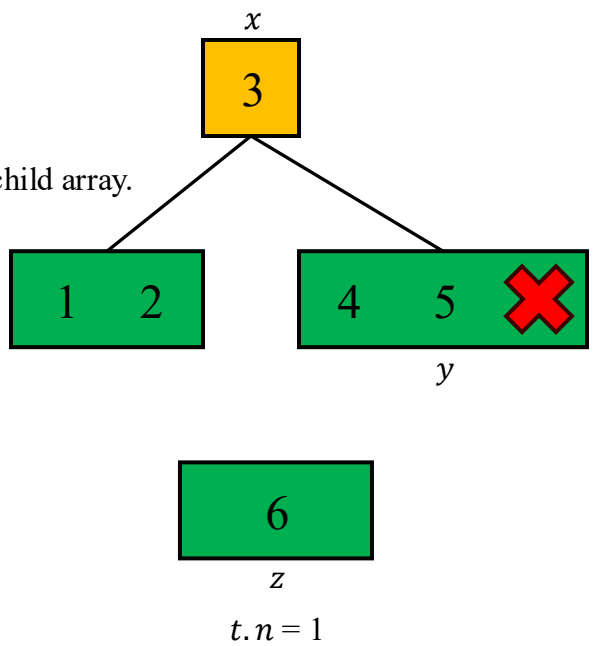
```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

Inserting z into parent's child array.

- $j = 1 + 1 = 2$.
- $i + 1 = 3$.



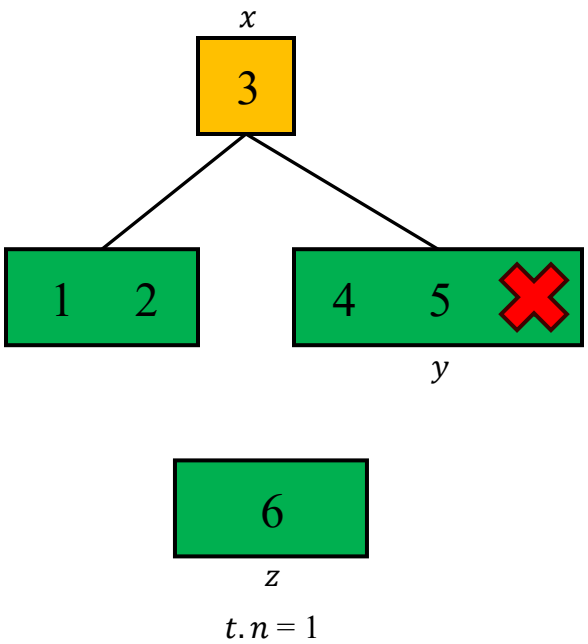
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

No shifting is necessary

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

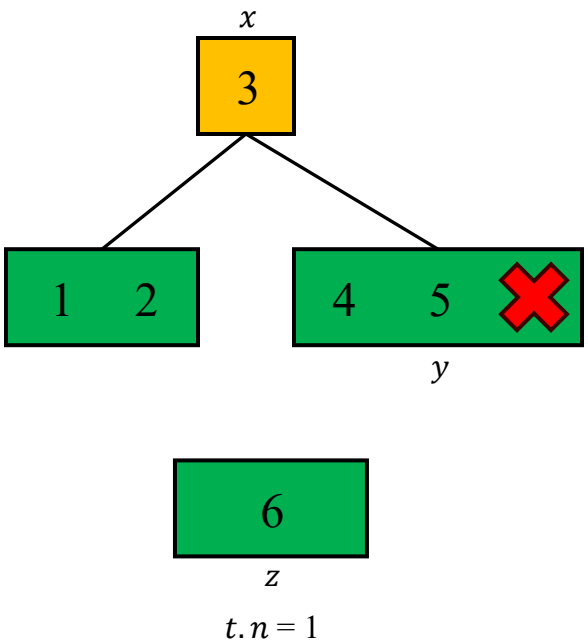


Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

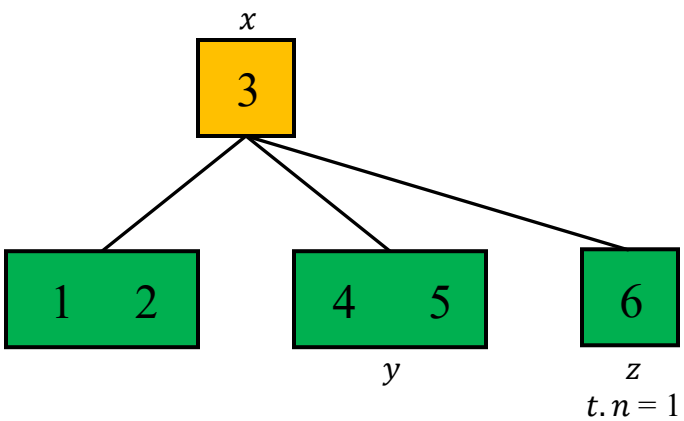


Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



Inserting 7 into a B-tree

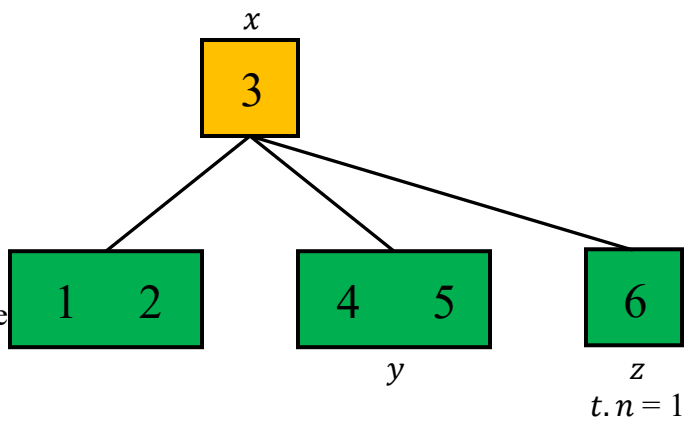
```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Adjust parent node keys and structure

- $x.n = 1$.
- $i = 2$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



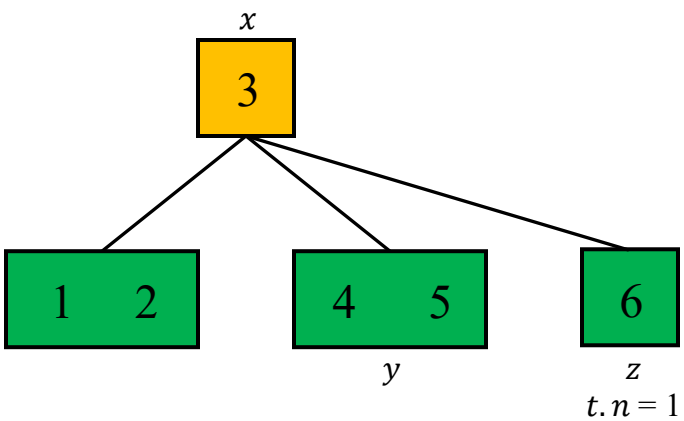
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$t = 2$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



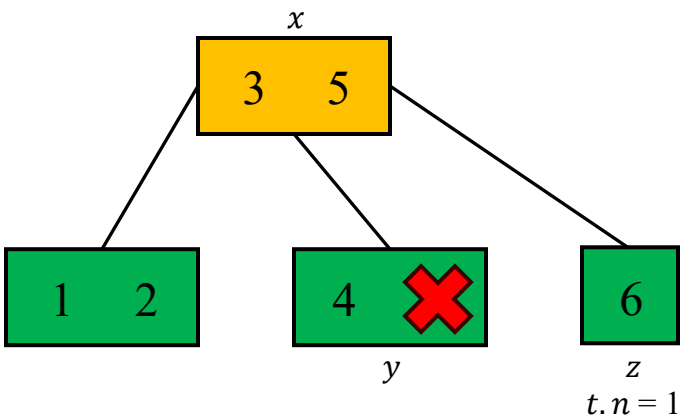
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$t = 2$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



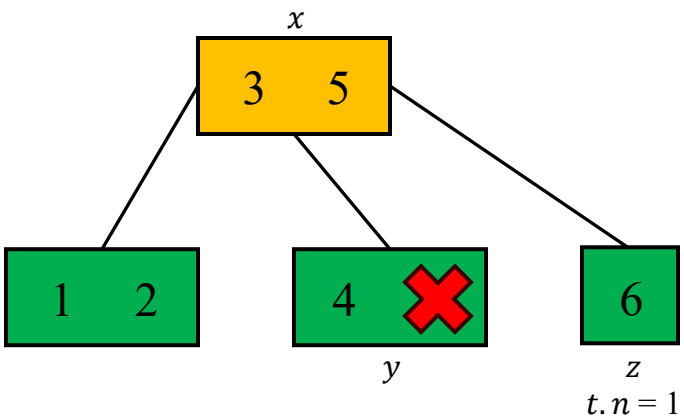
Inserting 7 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Save changes to y, z, and x to disk.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



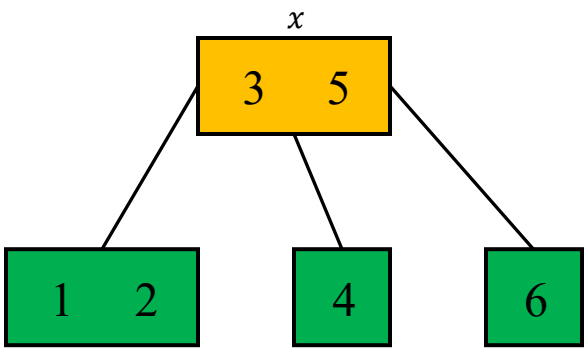
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$i = 2.\ 7 > x.key[2] = 5$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



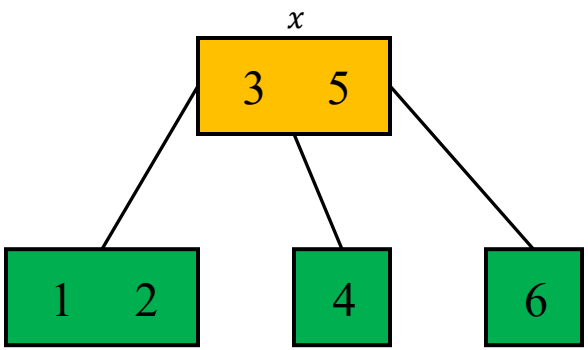
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$i = 3.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



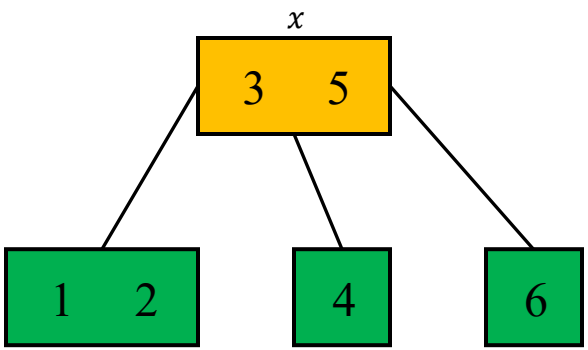
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

- $x.c_3 = [6]$.
- $k = 7$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



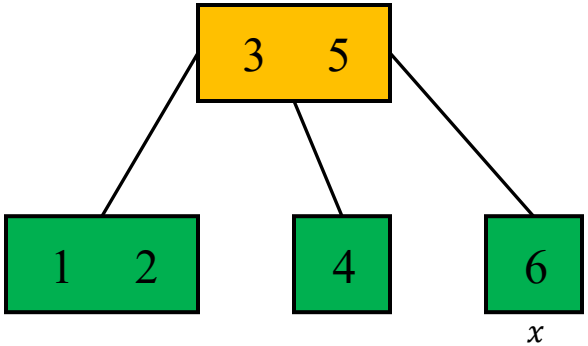
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x = [6]$ and $k = 7$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



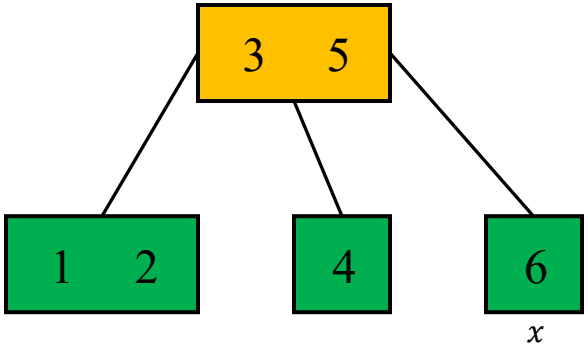
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$i = 1.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

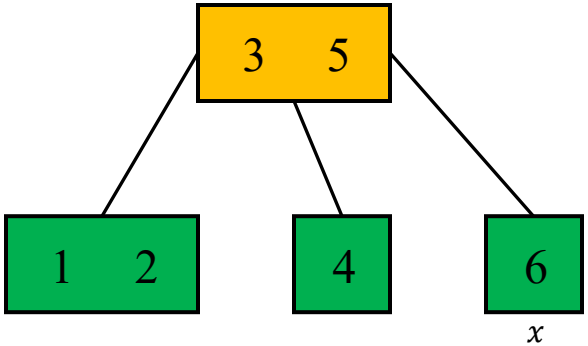


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf ← True.
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

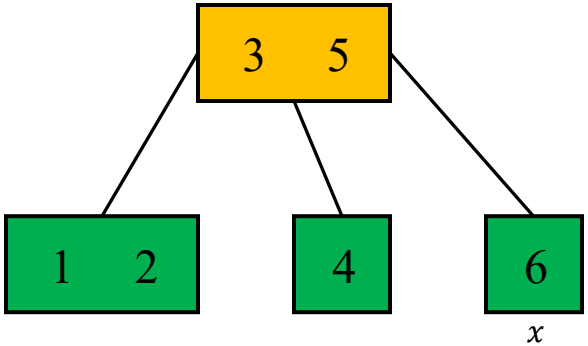


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



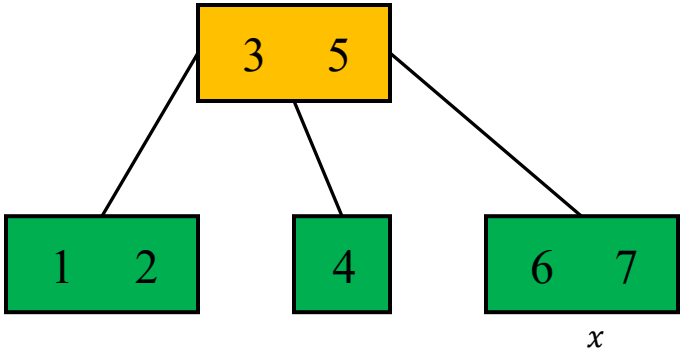
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.key_2 = 7$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$



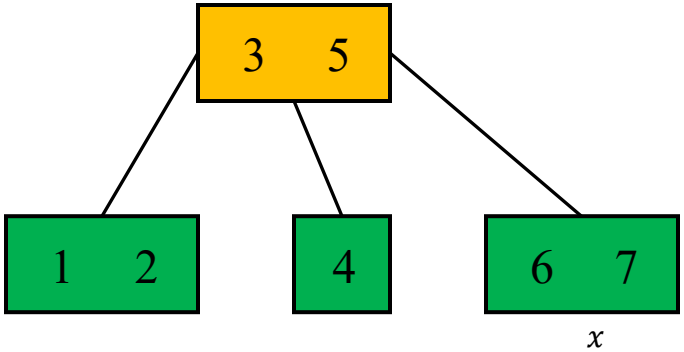
Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

$x.n = 2$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$

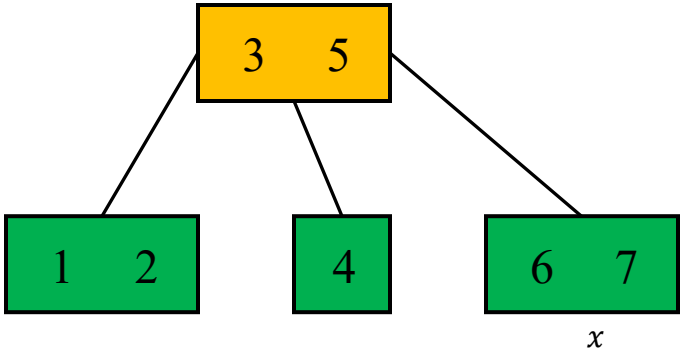


Inserting 7 into a B-tree

```
B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
  while i >= 1 and k < x.key_i
    x.key_(i+1) = x.key_i
    i = i - 1
  x.key_(i+1) = k
  x.n = x.n + 1
  DISK-WRITE(x)
else
  while i >= 1 and k < x.key_i
    i = i - 1
  i = i + 1
  DISK-READ(x.c_i)
  if x.c_i.n == 2t - 1
    B-TREE-SPLIT-CHILD(x, i)
    if k > x.key_i
      i = i + 1
  B-TREE-INSERT-NONFULL(x.c_i, k)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

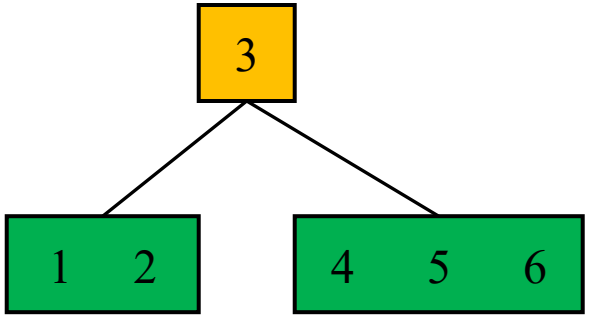
$k = 7$

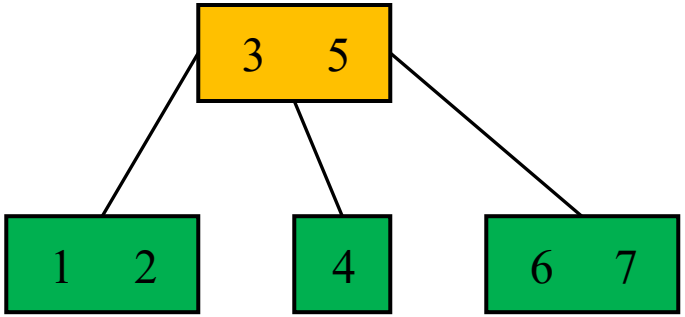


Inserting 7 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 7$





Inserting 10 into a B-tree

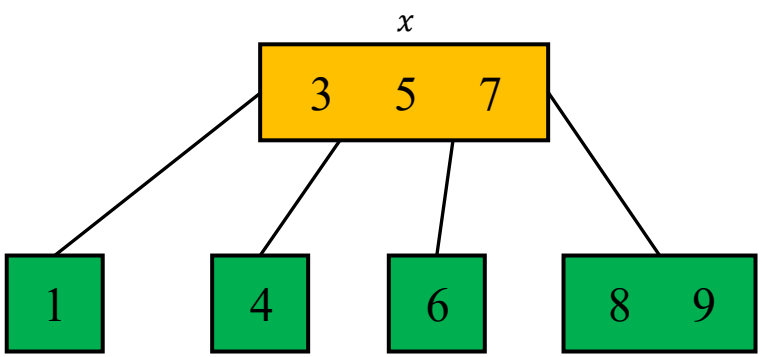
***Focusing on B-Tree-Split-Root(T).

Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

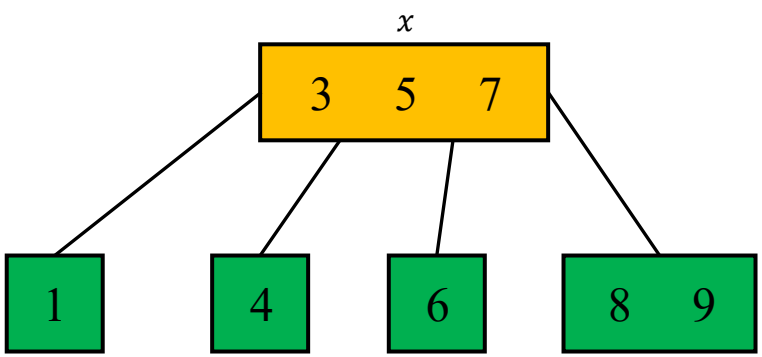


Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

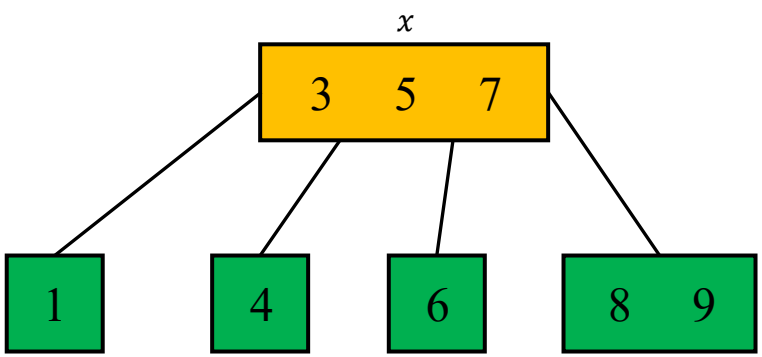


Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```



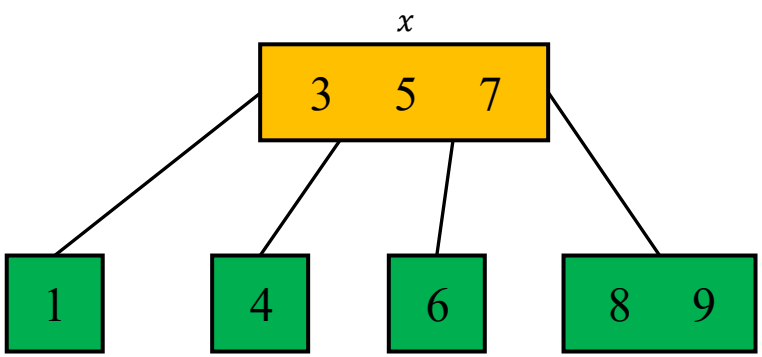
Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

$r.n = 3$

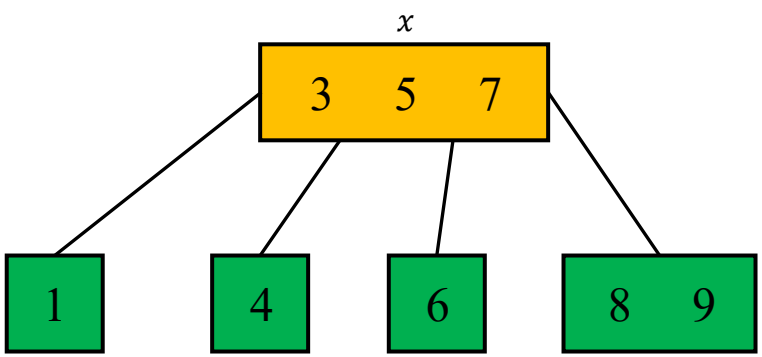


Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```

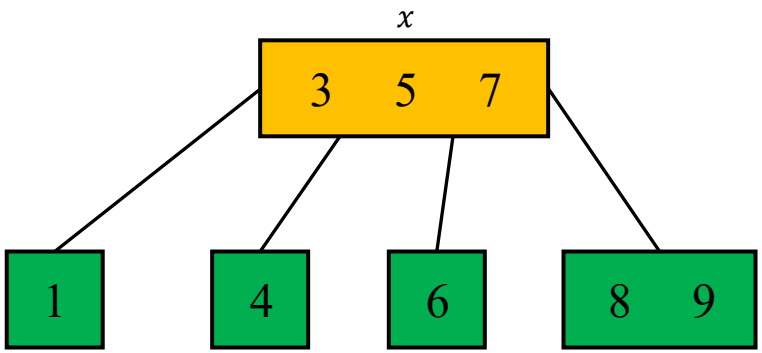


Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-SPLIT-ROOT(T)
  s = ALLOCATE-NODE()
  s.leaf = FALSE
  s.n = 0
  s.c1 = T.root
  T.root = s
  B-TREE-SPLIT-CHILD(s, 1)
  return s
```

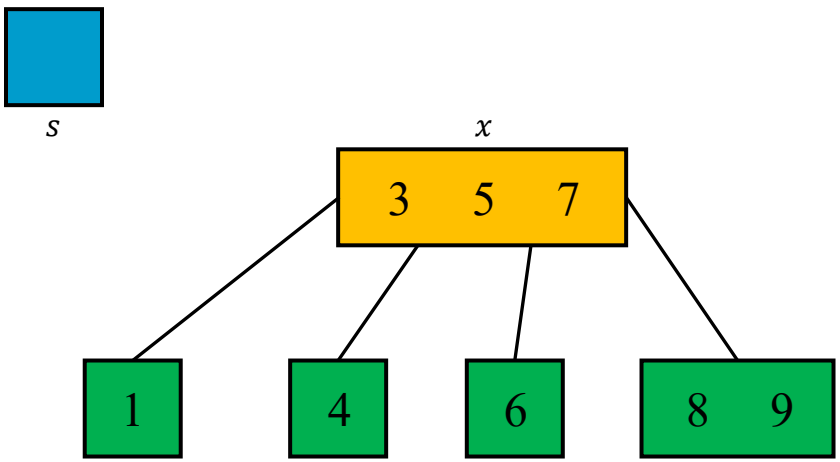


Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



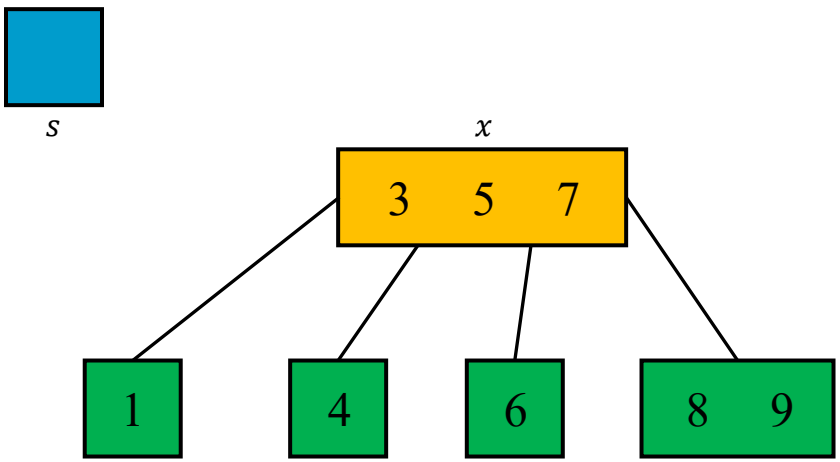
Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

s is an internal node.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

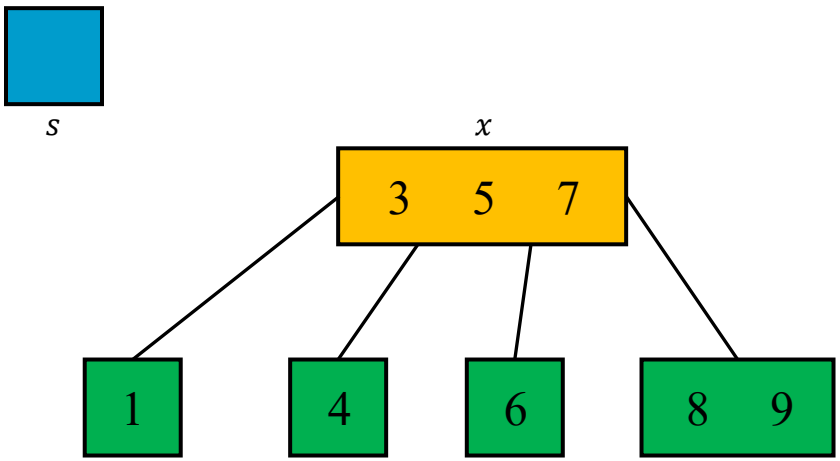


Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

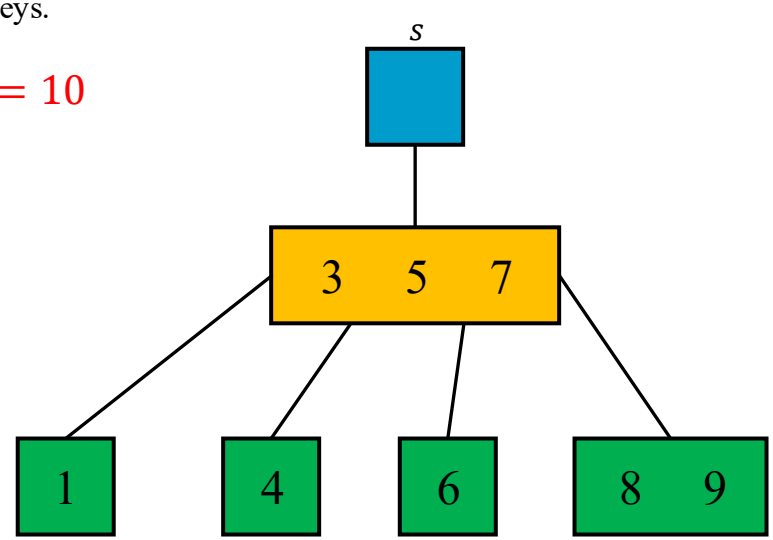


Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

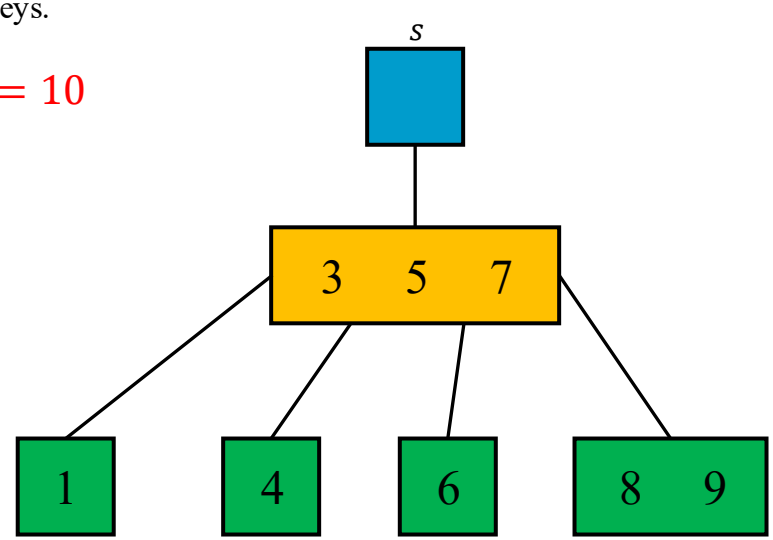


Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

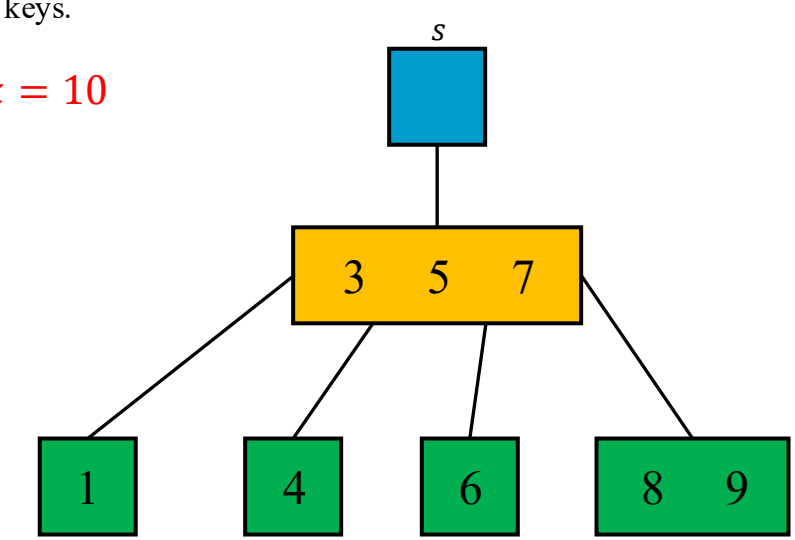


Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



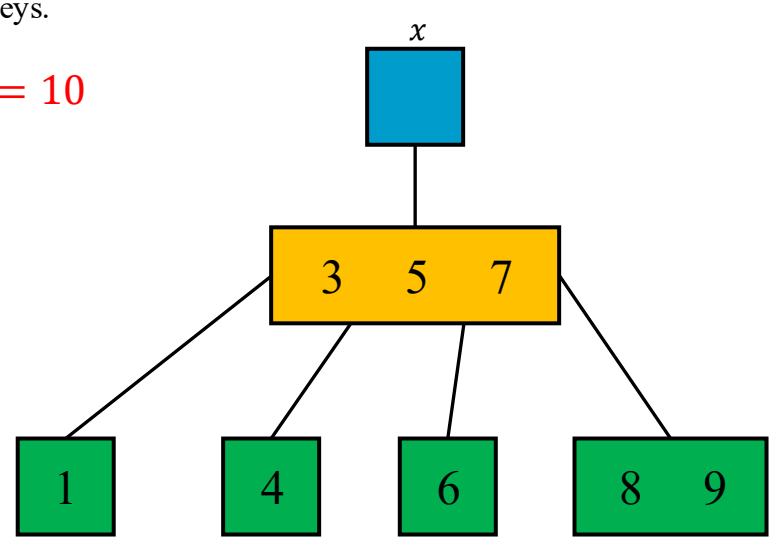
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$i = 1.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



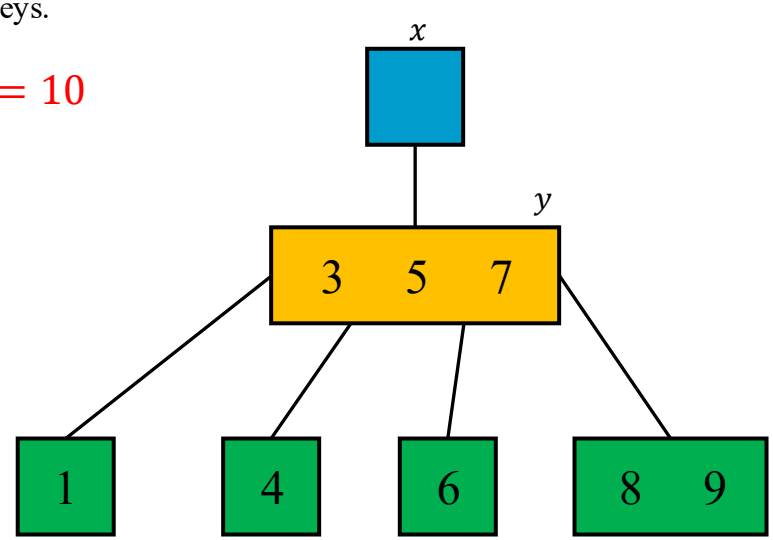
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$y = x.c_1 = [3, 5, 7]$.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

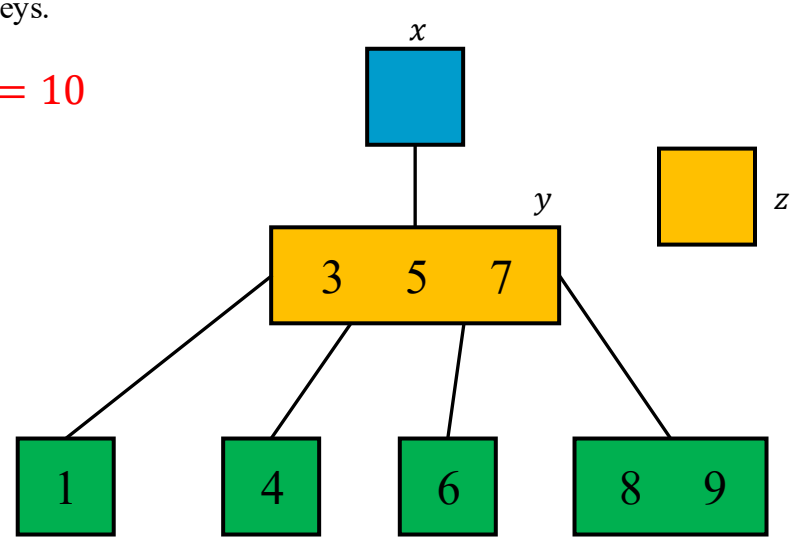


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



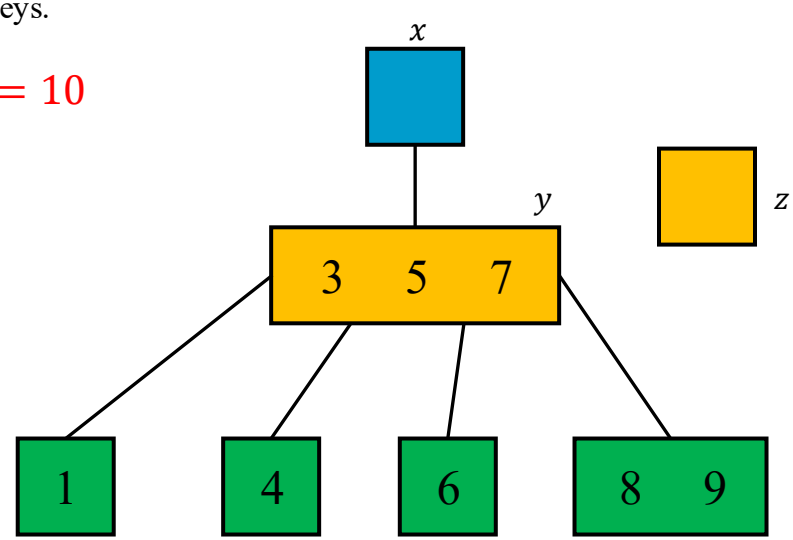
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

y.leaf is an internal node.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



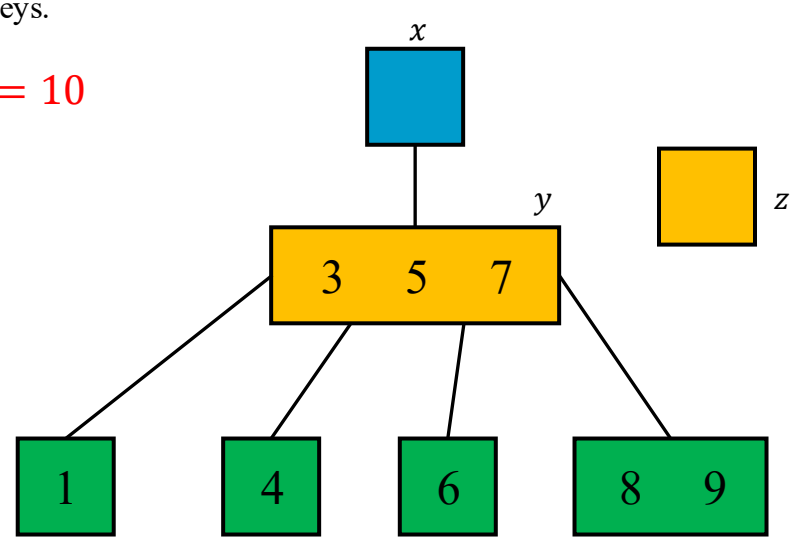
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$z.n = 2 - 1 = 1.$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



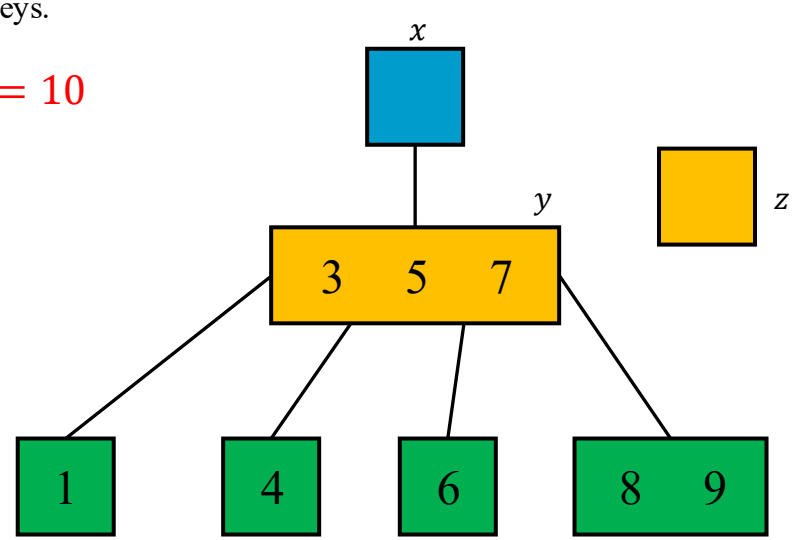
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Key Movement.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



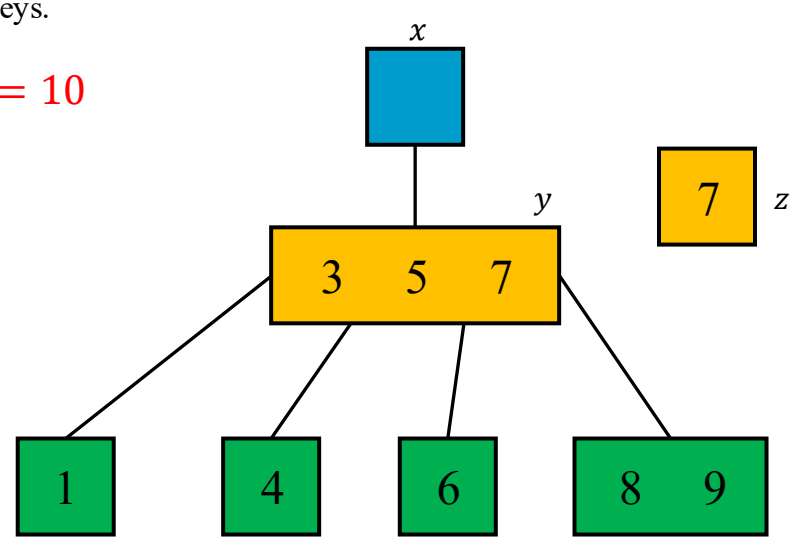
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

$y.key_3 = 7$

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

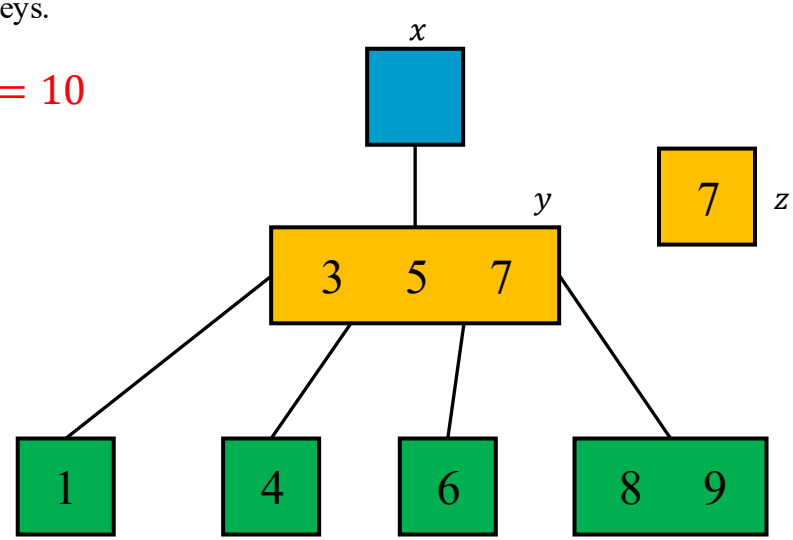


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

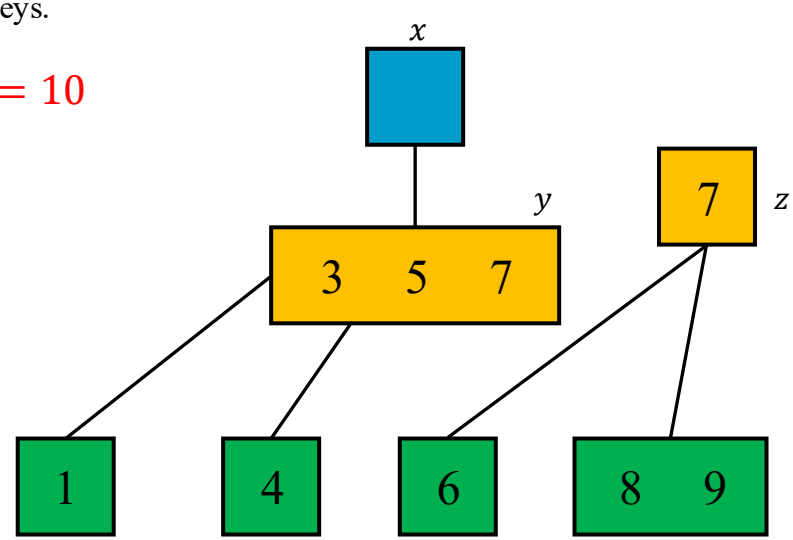


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

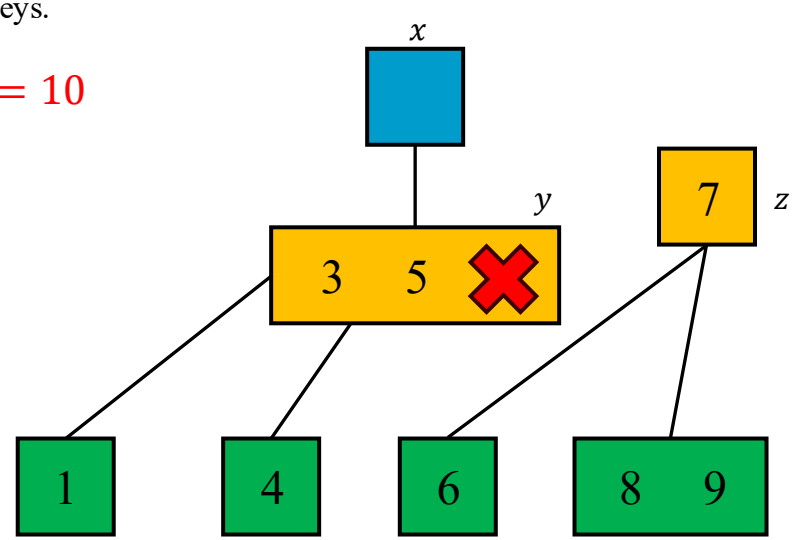


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

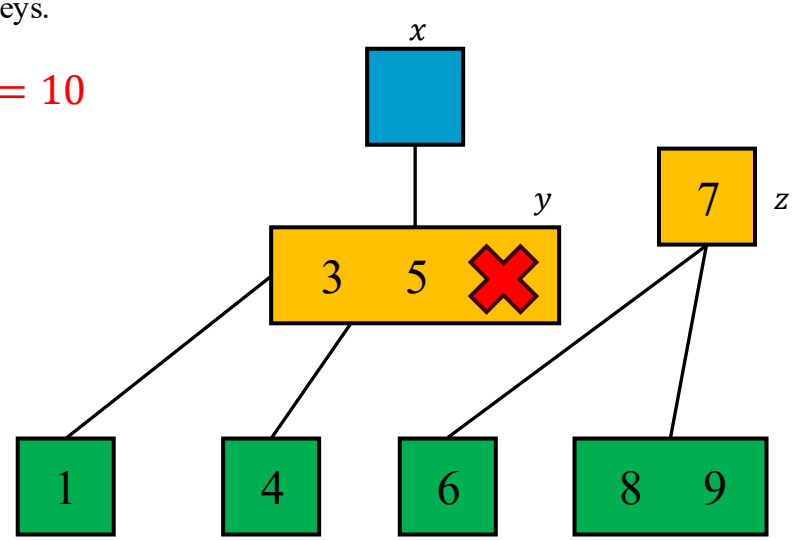


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

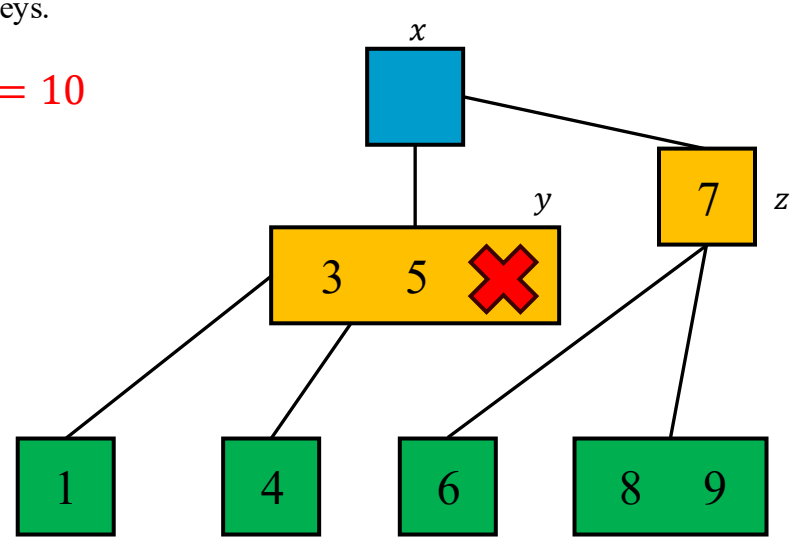


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

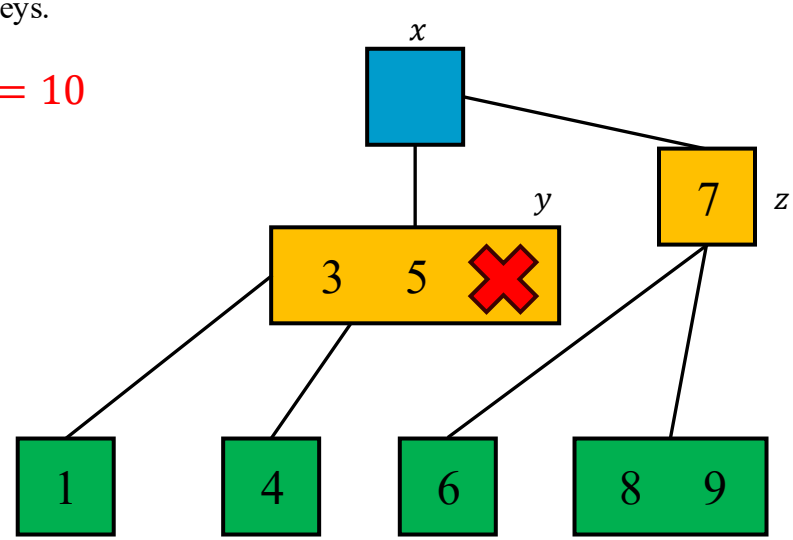


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

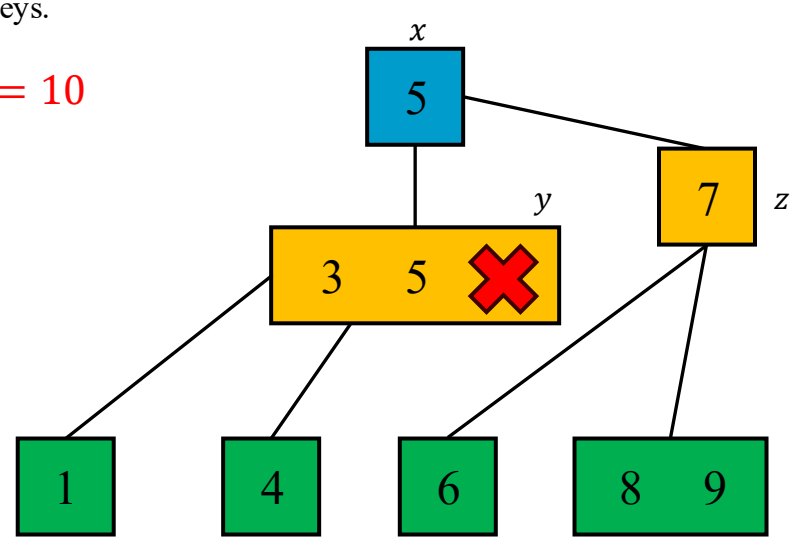


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

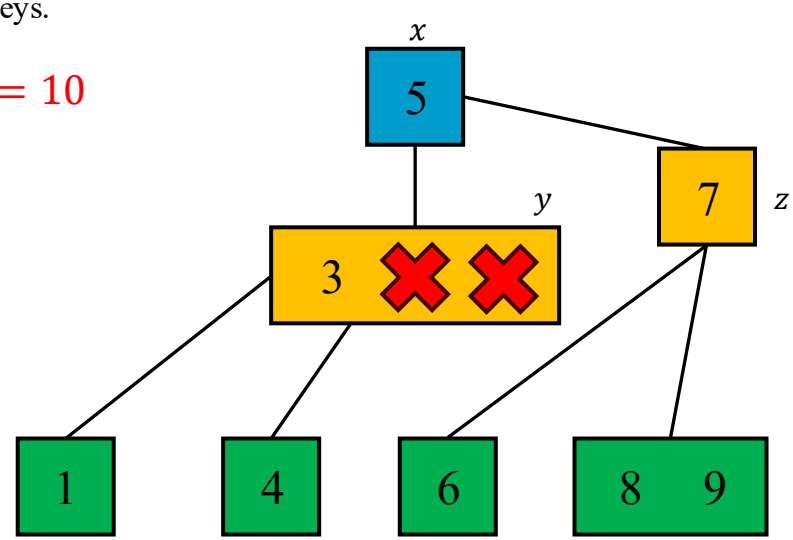


Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$



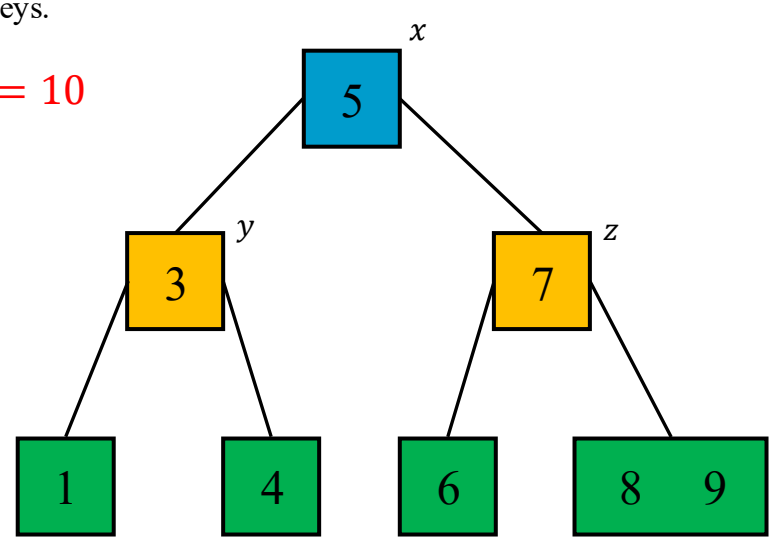
Inserting 10 into a B-tree

```
B-TREE-SPLIT-CHILD(x, i)
y = x.c_i
z = ALLOCATE-NODE()
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
  z.key_j = y.key_(j+t)
if not y.leaf
  for j = 1 to t
    z.c_j = y.c_(j+t)
y.n = t - 1
for j = x.n + 1 downto i + 1
  x.c_(j+1) = x.c_j
x.c_(i+1) = z
for j = x.n downto i
  x.key_(j+1) = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

Save changes to y, z, and x to disk.

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

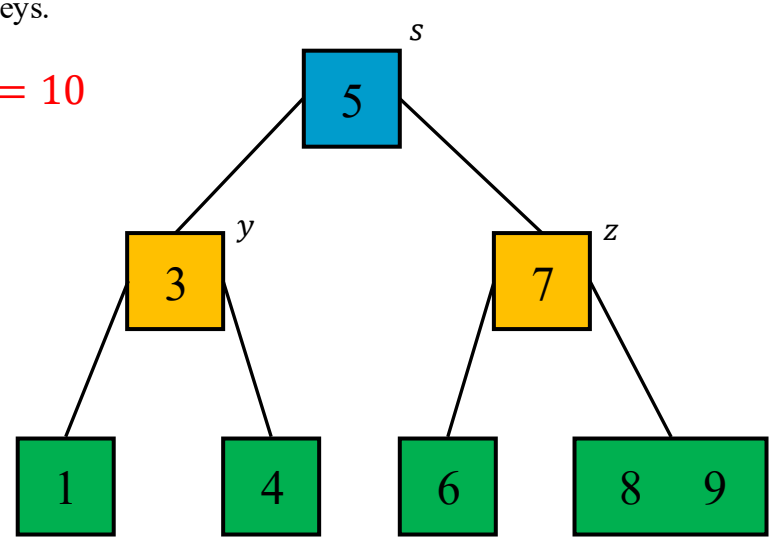


Inserting 10 into a B-tree

```
B-TREE-SPLIT-ROOT(T)
s = ALLOCATE-NODE()
s.leaf = FALSE
s.n = 0
s.c1 = T.root
T.root = s
B-TREE-SPLIT-CHILD(s, 1)
return s
```

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

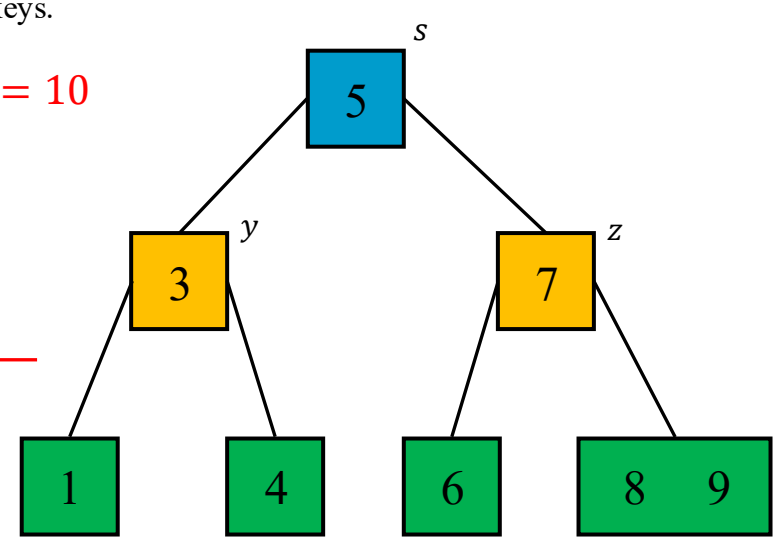


Inserting 10 into a B-tree

Let's consider a B-tree with a minimum degree $t = 2$, which means each node can have a minimum of 1 and a maximum of 3 keys.

$k = 10$

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t - 1
        s = B-TREE-SPLIT-ROOT(T)
        B-TREE-INSERT-NONFULL(s, k)
    else B-TREE-INSERT-NONFULL(r, k)
```



B-Tree Practice from the Textbook.

Splitting the root r with $t = 4$

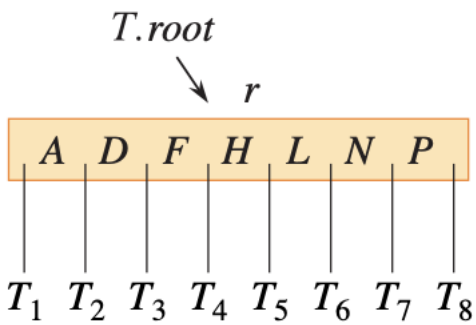


Figure: Splitting the root with $r = 4$.

Splitting the root r with t = 4

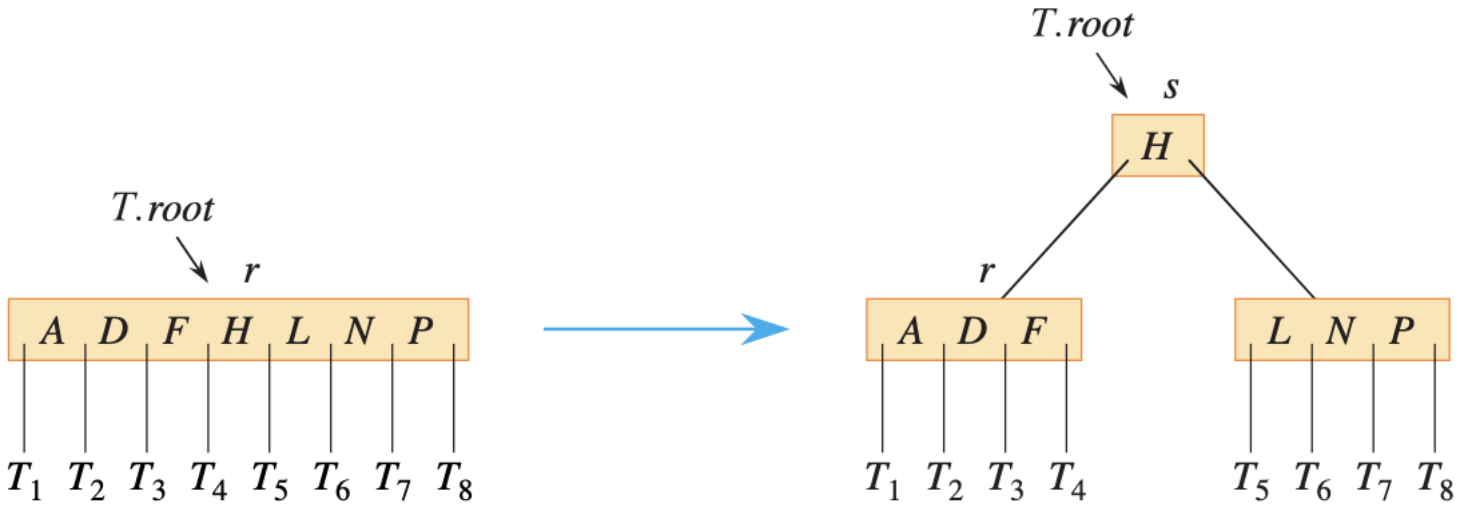
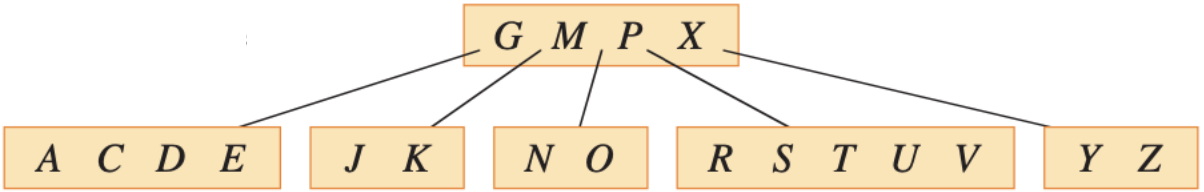


Figure: Splitting the root with r = 4.

***The new root contains the median key of r and has the two halves of r as children.

Inserting keys into a B-tree

***The minimum degree for this B-tree is 3.

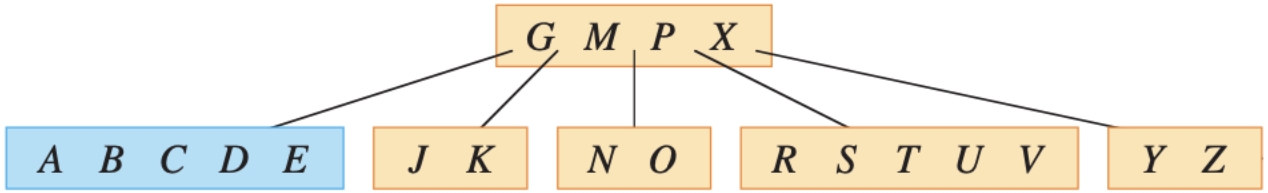


(a) Initial Tree.

Insert B.

Inserting keys into a B-tree

***The minimum degree for this B-tree is 3.

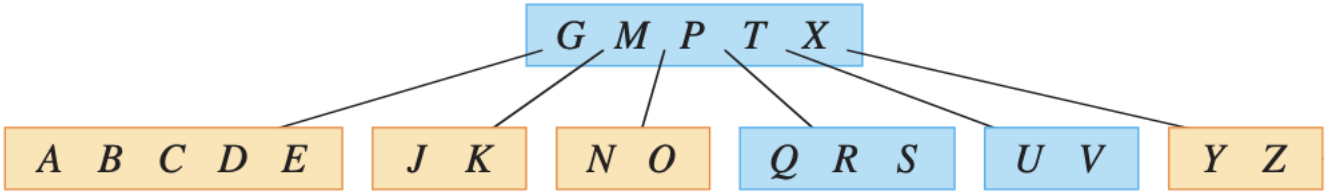


(b) B Inserted.

Insert Q.

Inserting keys into a B-tree

***The minimum degree for this B-tree is 3.

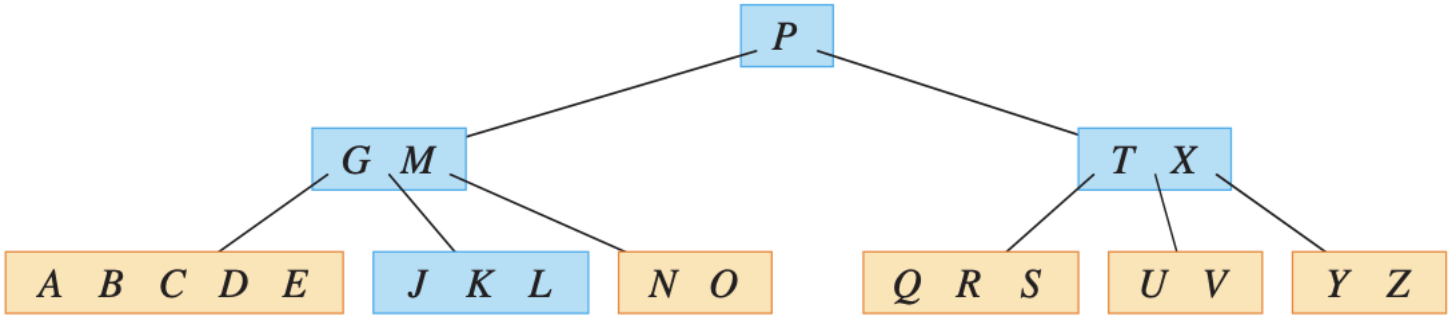


(c) Q Inserted.

Insert L.

Inserting keys into a B-tree

***The minimum degree for this B-tree is 3.

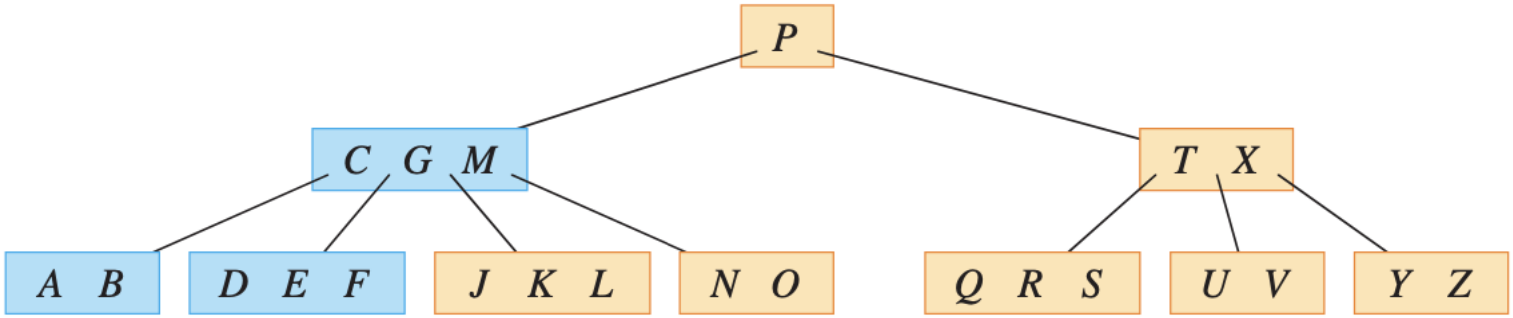


(d) L Inserted.

Insert F.

Inserting keys into a B-tree

***The minimum degree for this B-tree is 3.



(e) *F* Inserted.

Questions?

ROLL TIDE®